

UNIVERSITY OF TRIESTE

Department of Mathematics and Geoscience



Master's Degree in
Data Science and Artificial Intelligence

**How LLaVA Learns to See:
Localizing Representation Changes in
Visual Instruction Tuning**

September, 2025

Candidate
Luis Fernando Palacios Flores

Supervisors
Dr. Alberto Cazzaniga¹
Dr. Diego Doimo¹

Co-supervisor
Dr. Lorenzo Basile¹

¹ AREA Science Park

Dedication

To every dissatisfied soul that does not stop its movement.

Acknowledgments

First and foremost, I am deeply grateful to SISSA and Collegio Fonda for supporting me financially and giving me a place to live during my master's degree. The lectures at SISSA, together with its inspiring environment and views, made learning a real pleasure. Collegio Fonda became my home for these two years, providing me with the space and calm I needed to fully immerse myself in my studies.

I would like to sincerely thank Dr. Alberto Cazzaniga for his guidance and support throughout this research and the final stages of my master's degree. His expertise, enthusiasm, great vibe and approachable nature made a lasting impact on my work and my way of thinking.

I am also grateful to Dr. Diego Doimo for his guidance, thoughtful suggestions and feedback, which greatly improved the quality of this thesis, and to Dr. Lorenzo Basile for insightful discussions, readiness to help, and steadfast support during the execution of this work.

I learned a great deal during this master's thesis. Thanks you so very much, Alberto, Diego, and Lorenzo.

My thanks also go to Francesco Ortu and Alessandro Serra for helpful conversations during the development of this work.

I am thankful to the Laboratory of Data Engineering at AREA Science Park for the supportive and welcoming environment I experienced during this thesis.

I also want to thank the University of Trieste for giving me the opportunity to study a subject I am passionate about.

Finally, my deepest gratitude goes to my family and friends for their constant encouragement and emotional support throughout this journey far from home.

Abstract

LLaVA is a pioneering vision-language model that grounds a large language model (LLM) in images to enable multimodal assistance. This thesis investigates how visual instruction tuning reshapes the backbone’s *representations geometry*, where along the network’s depth these changes concentrate, and how they relate to downstream performance. We analyze all backbone layers using layer- and head-wise (within attention layers) representation similarity, profile the intrinsic dimension, and evaluate the impact of layer patching. Our results show that the effects of visual instruction tuning localize to the early-mid layers, where the similarity between the pretrained and fine-tuned models drops the most, especially for multimodal inputs. The same region shows strong geometric compression (lower intrinsic dimension), and cumulatively patching early-mid layers from the fine-tuned checkpoint into the pretrained one recovers most gains on visual question answering and image captioning, whereas patching late layers yields diminishing returns. Overall, visual instruction tuning teaches LLaVA to *see* by primarily modifying the early-mid region of the network, while late layers act as largely modality-agnostic decoders—suggesting targeted, cost-effective fine-tuning strategies and clarifying how multimodal LLMs integrate modalities. The code for this thesis is available at https://github.com/luispky/LLaVA_Representations_Analysis.git.

Abstract (Italian)

LLaVA è un modello di visione e linguaggio pionieristico, che collega un *large language model (LLM)* alle immagini per abilitare l'assistenza multimodale. Questa tesi analizza come il *visual instruction tuning* rimodella la geometria delle rappresentazioni del modello, in che punti della rete si concentrano questi cambiamenti e in che modo questi si riflettono sulle prestazioni a valle. Analizziamo tutti i *layer* del modello utilizzando misure di similarità tra rappresentazioni a livello di *layer* e di *head* (all'interno dei *layer* di attenzione), oltre a misurarne la dimensione intrinseca e valutare la possibilità di trasferire interi *layer*. I nostri risultati mostrano che gli effetti del visual instruction tuning si localizzano principalmente nella prima metà del modello, dove la similarità tra il modello originale e quello *fine-tuned* è più bassa, soprattutto per input multi-modali. La stessa regione mostra una forte compressione geometrica (dimensione intrinseca più bassa) e il trasferimento cumulativo dei primi *layer* dal modello *fine-tuned* in quello originale consente di recuperare gran parte dei miglioramenti in *visual question answering* e *image captioning*, mentre il trasferimento degli strati finali comporta rendimenti decrescenti. In sintesi, il visual instruction tuning insegna a LLaVA a vedere modificando principalmente la porzione iniziale della rete, mentre gli strati successivi agiscono in gran parte come decoder indipendenti dalla modalità—suggerendo strategie di fine-tuning mirate ed economicamente efficienti e chiarendo come gli LLM multi-modali integrino le diverse modalità. Il codice sviluppato per questa tesi è disponibile all'indirizzo https://github.com/luispky/LLaVA_Representations_Analysis.git.

Contents

Dedication	1
Acknowledgments	2
Abstract	3
Abstract (Italian)	4
1 Introduction	7
1.1 Context	7
1.2 Research Problem and Motivation	8
1.3 Contributions	8
1.4 Thesis Structure	9
2 Literature Review	10
2.1 Instruction-tuned Large Language Models	10
2.1.1 Transformer-based Language Models and the Residual Stream . .	10
2.1.2 Instruction-tuning and chatbots	12
2.2 Vision-Language Models	12
2.3 LLaVA	13
2.4 Mechanistic Interpretability	14
2.5 Representation Similarity and Internal Geometry	14
2.6 Cross-Modal Alignment and Modality Gap	15
3 Methodology	16
3.1 Datasets	16
3.1.1 Question Answering	16
3.1.2 Image Captioning	18
3.2 Residual Stream Extraction Method	19
3.3 Representational Similarity Measures	20
3.3.1 Neighborhood Overlap	21
3.3.2 Linear CKA	21
3.4 Geometry of Representations	21
3.4.1 Intrinsic Dimension of Representations	21

3.4.2	Modality Gap	22
3.5	Functional Localization via Layer Patching	25
3.6	Computational Resources and Reproducibility	26
4	Results and Discussion	27
4.1	Zero-shot QA: Performance Effects of Visual Instruction Tuning	27
4.2	Where the Backbone Changes: Representational Similarity	29
4.3	Geometry After Fine-Tuning: Intrinsic Dimension	32
4.4	Modality Gap After Fine-Tuning: Alignment vs. Separation	34
4.5	Task-Level Validation via Layer Patching	36
4.5.1	Question Answering	36
4.5.2	Image Captioning	36
5	Conclusion	39
5.1	Key findings	39
5.2	Limitations and Future Work	40
A	Appendix	41

Chapter 1

Introduction

1.1 Context

Artificial intelligence has rapidly advanced through Large Language Models (LLMs), which demonstrate remarkable abilities in natural language understanding, reasoning, and instruction following [1, 2]. These models have fundamentally changed how we think about AI systems, showing that scaling up neural networks trained on massive text corpora can unlock emergent capabilities ranging from code generation to complex reasoning.

Recently, researchers have extended this paradigm to create vision-language models that integrate text and vision, enabling systems that not only converse but also see [3, 4]. This represents a crucial step toward more general AI systems that can interact with the real world through multiple modalities. Visual-language assistants demonstrate that the reasoning capabilities of LLMs can be successfully grounded in visual perception, opening new possibilities for AI applications.

Understanding how these models integrate modalities is crucial not only for improving performance, but also for ensuring robustness, transparency, and trustworthiness. As these systems become more capable, we need to understand their internal mechanisms to ensure they behave reliably and can be debugged when they fail.

LLaVA (Large Language and Vision Assistant) [5, 6] represents a breakthrough in vision-language modeling: a vision encoder feeds a projector that maps visual features into an LLM backbone (see Figure 1.1), enabling image-grounded dialogue after instruction tuning. LLaVA acquires its visual instruction-following ability in two stages: (i) a projection layer is learned to align vision features with the LLM’s embedding space while leaving the backbone unchanged; (ii) *visual instruction tuning* fine-tunes the projector and backbone (with the vision encoder frozen) on image-text instruction-response pairs, equipping the system to follow image-grounded instructions. While the success of LLaVA on visual instruction following benchmarks is well-documented [5, 6], we lack a precise account of where and how visual instruction tuning alters the LLM backbone itself to enable LLaVA to follow image-grounded instructions, and whether these changes differ between text-only and multimodal inputs.

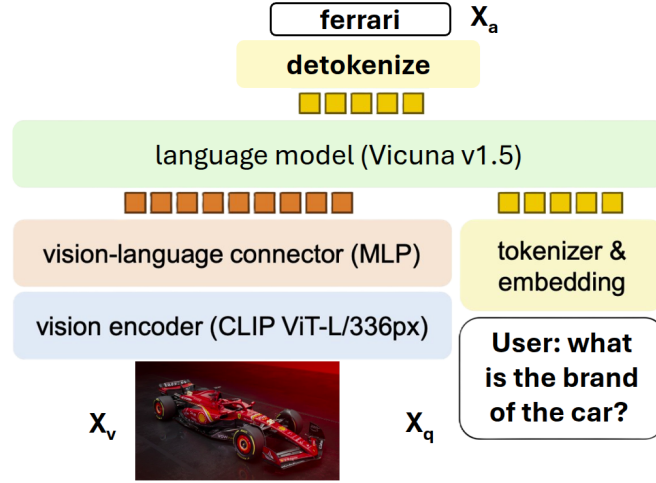


Figure 1.1: LLaVA v1.5 architecture. The model consists of a vision encoder (CLIP-ViT-L-336px) [7], a language model backbone (Vicuna v1.5) [8], and an MLP projection layer. Adapted from [6].

1.2 Research Problem and Motivation

While the alignment stage updates only the projection layer, the fine-tuning stage modifies the language-model backbone itself. This two-stage training setup raises a fundamental question:

How does visual instruction fine-tuning change the internal representations of LLaVA’s backbone, and where in the depth of the network do these changes concentrate?

This thesis addresses this question.

From an interpretability perspective, it is crucial to identify which parts of the backbone support visual reasoning and how they change under fine-tuning. In practice, this understanding enables performance improvements while preserving core language abilities and enhancing robustness across tasks and inputs. Localizing where changes concentrate enables more efficient training such as targeting specific layers instead of end-to-end updates. Ultimately, understanding how internal representations develop is essential for safe and reliable deployment on downstream tasks.

1.3 Contributions

This work makes four key contributions to the understanding of the impact of vision instruction fine-tuning in LLaVA:

1. **Layer-localized representational analysis:** Through representational similarity measures such as Neighborhood Overlap and Linear CKA, we quantify that the

LLaVA backbone changes most in the early-mid layers with partial convergence to the equivalent representational similarity between text-only and multimodal inputs in deep layers in question answering tasks.

2. **Geometric characterization:** We compare the geometry of the representations before and after fine-tuning and demonstrate that fine-tuning compresses mid-layer geometry (lower intrinsic dimension and entropy), especially for multimodal inputs.
3. **Modality-gap analysis:** Given the importance of textual and visual modality integration in multimodal models for tasks such as multimodal retrieval, we measure how the geometric separation between text and image representations or *modality gap* evolves with depth and how fine-tuning shifts that gap across different tasks.
4. **Functional validation via layer patching:** We show that patching early-mid fine-tuned layers into the pretrained backbone recovers most performance gains, while late layers yield diminishing returns.

1.4 Thesis Structure

The remainder of the thesis is organized as follows. Chapter 2 provides a review on instruction-tuned LLMs, vision-language models, the LLaVA architecture and its training paradigm, and the theoretical foundations for our analysis methods. Chapter 3 details our experimental approach, including datasets, similarity measures, geometric analysis tools, and the layer patching protocol. Chapter 4 presents our findings on representation similarity, geometric reorganization, modalities alignment, and functional validation through layer patching. Chapter 5 summarizes our key findings, discusses their implications, acknowledges limitations, and outlines directions for future research.

Chapter 2

Literature Review

2.1 Instruction-tuned Large Language Models

2.1.1 Transformer-based Language Models and the Residual Stream

The transformer architecture [9] revolutionized natural language processing by modeling long-range dependencies more effectively than earlier architectures. This breakthrough paved the way for the era of foundation models, where large-scale transformers trained on massive, diverse datasets became general-purpose architectures adaptable to a wide range of downstream tasks [10]. Originally designed for sequential data such as text or speech, this neural network architecture relies on *self-attention* [11, 12], which allows the model to weigh different parts of the input sequence when processing each token. By flexibly attending to different parts of the input sequence, transformers learn richer representations [13], capture context more effectively, and generate more accurate predictions.

Transformers do not process raw text directly. Instead, text is decomposed into tokens—units such as words, subwords, or characters—using a tokenizer. A common scheme is Byte Pair Encoding (BPE) [14]. The tokenizer is trained on a large corpus and builds a vocabulary $\mathcal{V}$ of size V , where each token is mapped to a unique integer index $t_i \in 1, \dots, V$. Given a string of text, the tokenizer converts it into a sequence $t_1, \dots, t_T$ that the model can process.

A language model is trained to predict the next token given the preceding ones. Formally, for a sequence of tokens $(t_1, \dots, t_T)$, the joint probability is factorized using the chain rule of probability:

$$P(t_1, \dots, t_T) = \prod_{i=1}^T P(t_i \mid t_1, \dots, t_{i-1}).$$

Training maximizes the log-likelihood of the observed tokens over a large corpus:

$$\mathcal{L} = - \sum_{i=1}^T \log P_{\theta}(t_i \mid t_{<i}),$$

where θ denotes the model parameters. This objective ensures the model learns to generate text autoregressively, one token at a time.

The decoder-only transformer, used for text generation, consists of a stack of L blocks (Figure 3.2). Each block has two main components: a masked self-attention layer followed by a multi-layer perceptron (MLP). Input tokens are first mapped into a d -dimensional embedding space by a learnable projection:

$$e_i = W_{\text{emb}} t_i, \quad e_i \in \mathbb{R}^d,$$

where $W_{\text{emb}} \in \mathbb{R}^{d \times V}$ is the embedding matrix. These embeddings serve as the initial hidden states or representations:

$$h_i^{(0)} = e_i, \quad i = 1, \dots, T.$$

Across the L layers, the hidden states are repeatedly transformed but remain in $\mathbb{R}^d$. After the final block, the representation of the last token $h_T^{(L)}$ is projected back into vocabulary space to predict the next token:

$$z = W_{\text{out}} h_T^{(L)}, \quad P(t_{T+1} \mid t_{\leq T}) = \text{softmax}(z) \in \mathbb{R}^V,$$

where $W_{\text{out}} \in \mathbb{R}^{V \times d}$ maps the hidden state to vocabulary logits and the softmax produces a probability distribution over all tokens $t \in \mathcal{V}$.

During generation, the model can then select the most likely next token,

$$\hat{t}_{T+1} = \arg \max_{t \in \mathcal{V}} P(t \mid t_{\leq T}),$$

or sample from this distribution. The newly generated token $\hat{t}_{T+1}$ is appended to the sequence, and the process repeats, ensuring each prediction conditions on *all* previously generated tokens.

The Residual Stream. A key design feature of the transformers architecture is the residual connection [15], which guided the dimensional invariant feature of representations in the architecture. Instead of replacing information at each layer, residual connections preserve and accumulate it, forming a continuous *residual stream* [16]—a single representational backbone that “flows” through the entire network, continuously updated but never replaced.

Within each block, the residual stream is updated twice: first by attention, then by the feed-forward MLP. In practice, modern decoder-only transformers such as LLaMA [17] adopt a *pre-normalization* (pre-norm) design, where each transformation is applied to a normalized hidden state. Specifically, a Layer Normalization [18, 19] method is applied before both the attention and the feed-forward sublayers. For input $h^{(\ell)}$ to block ℓ , the residual stream is updated as

$$\begin{aligned} h'^{(\ell)} &= h^{(\ell)} + \text{Attn}(\text{LayerNorm}(h^{(\ell)})), \\ h^{(\ell+1)} &= h'^{(\ell)} + \text{MLP}(\text{LayerNorm}(h'^{(\ell)})). \end{aligned}$$

This design has two consequences. First, it stabilizes gradient flow and eases optimization in deep networks by providing direct paths for information and gradients. Second, it enables later layers to refine earlier representations rather than overwrite them, ensuring information persists across depth. Figure 3.2 illustrates this pre-norm transformer decoder block, the backbone of LLaVA. A sequence of embeddings $x \in \mathbb{R}^{T \times d}$ flows through the residual stream, contextualized by attention, refined by the MLP, and accumulated across layers into a persistent d -dimensional representation.

2.1.2 Instruction-tuning and chatbots

Pretraining decoder-only LLMs purely with next-token prediction yields fluent, context-consistent continuations, but does not by itself confer the ability to follow human instructions or behave as interactive assistants or *chatbots*. Instruction tuning [20–22] adapts these models by fine-tuning on instruction-response pairs, often augmented with human preference signals, so they reliably follow natural-language prompts. A landmark example is InstructGPT [22]: aligning GPT-3 [23] with human feedback substantially improved helpfulness and factuality, with a 1.3B model preferred over a 175B baseline on instruction-following tasks. This paradigm underlies modern conversational systems (e.g., ChatGPT) and has been widely adopted in open-source efforts; for instance, *Vicuna* [8] fine-tunes LLaMA-family checkpoints on ShareGPT dialogue data, reports strong instruction following capabilities, and serves as the conversational backbone for several multimodal assistants [5, 6, 24].

2.2 Vision-Language Models

Vision-language models (VLMs) extend large language models by incorporating visual inputs alongside text, enabling them to jointly process and reason over multimodal information. Language models alone are blind, vision models alone are mute, but VLMs can do both and reason about images. The core idea is to project both images and text into a shared representation space where cross-modal alignment can occur. Typically, this is achieved by pairing a vision encoder (such as a convolutional network or vision transformer) with a language backbone, and then training the system so that semantically related images and text are close in the embedding space [3–7, 25]. This alignment allows the model to perform tasks such as image captioning, visual question answering, and image retrieval, where understanding both modalities is essential.

Recent advances build on this foundation by tightly coupling vision and language components through joint training and instruction tuning. Models such as CLIP [7] demonstrated the power of large-scale contrastive training for learning general-purpose visual-text embeddings. Newer systems such as LLaVA [5, 6], BLIP-2 [25], and MiniGPT-4 [24] adapt pretrained LLMs and vision models (e.g., ViT [26]) with vision adapters or lightweight projection layers. These approaches leverage the reasoning abilities of LLMs while grounding them in visual content, producing multimodal assistants capable of following natural-language instructions about images.

Other paradigms, such as Flamingo [27] and Emu3 [28], move beyond lightweight adapters by directly unifying vision and language through cross-attention layers or joint multimodal pretraining. This allows models to handle interleaved image-text inputs and generalize across a broader set of multimodal tasks.

2.3 LLaVA

LLaVA [5, 6] (Large Language and Vision Assistant) is one of the pioneering vision-language models developed as a general-purpose multimodal assistant, able to process and reason over both text and images. Its architecture combines three components: (i) a pretrained vision encoder (e.g., CLIP-ViT-L-336px in LLaVA v1.5); (ii) a pretrained large language model (e.g., Vicuna v1.5); and (iii) a multimodal connector, typically a two-layer MLP, that projects visual features into the token embedding space of the LLM. This architecture enables the model to process both text and images through a unified interface, following natural language instructions about visual content.

Unlike earlier task-specific systems for captioning or visual question answering [29–33], LLaVA was trained to follow natural-language instructions grounded in visual input. A key component for training LLaVA was a synthetically curated multimodal instruction-following dataset generated with GPT-4. Starting from open-source corpora such as COCO [34] and CC3M [35], the authors expanded simple image-text pairs into instruction-response conversations. Each instance follows the format (Figure 1.1): **User**: $X_v X_q$ <STOP> **Assistant**: X_a <STOP>, where X_v is the image, X_q the question, and X_a the assistant’s answer. <STOP> marks the end of the conversation turn. The model is trained to predict X_a given X_q and X_v , and to determine where each conversation turn ends. An additional **System** message is also prepended to enforce the instruction-following style. GPT-4 curated the answers to include richer descriptions, dialogue structure, and multi-step reasoning. This design allowed LLaVA to develop multimodal conversational ability with relatively modest data and compute, while still achieving competitive benchmark results. Like other assistant-style models, LLaVA relies on a chat template that structures input as **System**, **User**, and **Assistant** roles, ensuring compatibility with instruction-following training.

Training proceeds in two stages:

- **Feature Alignment (pretraining)**: The multimodal connector is trained while the vision encoder and LLM remain frozen. CLIP encodes each image into feature vectors, which the connector maps into the d -dimensional token space of the LLM. This ensures visual inputs can be directly interpreted as embeddings by the language model. The multimodal instruction training data is expanded from image-caption pairs to instruction-response conversations.
- **Visual Instruction Tuning (fine-tuning)**: The multimodal connector and the LLM are then fine-tuned jointly, while the vision encoder stays frozen. Using richer multimodal instruction-response pairs, the model learns to follow prompts and generate answers explicitly grounded in the input image.

2.4 Mechanistic Interpretability

Mechanistic interpretability is the study of how neural networks implement specific algorithms internally by reverse-engineering their weights, neurons, and *circuits* [16]—or subnetworks of interacting components—into human-understandable mechanisms [36]. With the increasing complexity of foundation models, there is a growing need to uncover their internal dynamics and decision-making processes. A wide range of methods has been developed, some specific to LLMs, others tailored to VLMs, and some applicable to both [36, 37]. Mechanistic interpretability is fundamental not only for explaining model behavior but also for mitigating undesired behaviors, improving safety and privacy, enhancing robustness and generalization, informing model design and much more [37].

Recent work has applied these methods to investigate how LLaVA changes through fine-tuning and how its mechanisms compare to its LLM backbone. Yu and Ananiadou [38] compared LLaVA’s mechanisms for visual question answering (VQA) against text QA, showing that: (i) LLaVA VQA relies on a similar in-context learning strategy as Vicuna text QA; (ii) visual embeddings are highly interpretable when projected into the language embedding space; and (iii) visual instruction tuning enhances certain textual capabilities of the base LLM rather than merely adding new ones. Khayatan et al. [39] analyzed fine-tuning dynamics through concept-based probing, mapping hidden states to interpretable visual and textual concepts. They showed that fine-tuning reshapes the concept space: some concepts specialize, new ones emerge, and others vanish. Importantly, many of these changes can be modeled as a simple shift from the original representation space, enabling direct steering of outputs by editing internal representations.

2.5 Representation Similarity and Internal Geometry

Another important line of research for interpreting and improving neural networks (e.g., by fine-tuning) focuses on measuring similarity of neural networks and analyzing their internal representations [40]. This is a vast field which involves analyzing the geometry of the representation space of models through data and grounds the detection of real representation sameness with mathematical guarantees. One approach is representational similarity, which considers how activations of intermediate layers differ, and another one is functional similarity, which considers how models differ in their outputs [40]. In this thesis, we focus on a subset of representational similarity approaches—namely, neighborhood overlap [41], linear CKA [42] and intrinsic dimension [43]—which have been applied in prior work to quantify similarities and differences in neural network representations. Here we use it to analyze the changes in LLaVA after visual instruction fine-tuning through its data representations. A detailed description of their computation and use in our study is provided in Chapter 3.

Prior studies have applied these tools across CNNs and transformers to reveal how representation geometry evolves through depth. Doimo et al. [41] used neighborhood

overlap to show that deep CNNs learn through a non-smooth process, with an abrupt transition in neighborhood composition toward ground-truth classes in the final layers. Ansuini et al. [43] found that trained CNN representations lie on very low-dimensional manifolds: intrinsic dimension rises in early layers and then decreases in deeper layers, reaching its minimum in the last hidden layer. Extending these insights to transformers, Valeriani et al. [44] observed the same expand-then-contract trend in token dimensionality and reported that neighborhood structure stabilizes in later layers, indicating more semantic clustering. Doimo et al. [45] further showed that late-layer neighborhood composition in LLMs changes the most during fine-tuning compared to zero-shot, and—through density-based clustering—that few-shot representations initially form clear semantic clusters (e.g., by question type or topic). After supervised fine-tuning, these clusters become more mixed, while later layers develop distinct answer-class clusters (sharper modes aligned with target answers). Finally, Serra et al. [46] analyzed VLMs and found that image and text representations become more separated within the residual stream, highlighting modality-specific organization in multimodal models.

2.6 Cross-Modal Alignment and Modality Gap

Multimodal models combine representations from different modalities to solve a task [4]. In VLMs such as LLaVA, however, image and text embeddings often cluster separately rather than fully overlapping [47, 48]. Liang et al. [47] formally studied this *modality gap* in the context of the CLIP model [7], showing that image and text embeddings form distinct clusters with a measurable offset between their centroids. They concluded that this gap arises partly from initialization and is reinforced during contrastive training, which aligns matched pairs but does not explicitly intermix modalities. Interestingly, they found that a wider gap sometimes improves CLIP’s zero-shot accuracy and fairness metrics by preventing interference between modalities. This suggests the modality gap is not simply a training artifact but may represent an equilibrium that balances cross-modal alignment with in-modal discrimination. In this thesis, we study whether visual instruction tuning changes the modality gap of LLaVA’s internal representations, as part of addressing the broader question: *what is the impact of the fine-tuning stage on the internal representations of the LLM backbone?*

While the modality gap does not necessarily harm performance in CLIP, later research argued that reducing it can benefit tasks requiring closer cross-modal interchangeability, such as multimodal retrieval. Goel et al. [49] introduced CyCLIP, which augments the contrastive loss with regularizers to enforce symmetry between image and text embedding spaces. This reduced in-modal vs. cross-modal discrepancies, leading to more balanced multimodal representations, though the gap was not eliminated. More recently, Role et al. [48] applied spectral- and optimal transport-based approaches to further shrink the gap, reporting gains on several downstream benchmarks. Together, these works show that training-time and post-processing interventions can meaningfully alter cross-modal geometry.

Chapter 3

Methodology

3.1 Datasets

In this work, we use question-answering and image captioning tasks to analyze the representations of pretrained and fine-tuned LLaVA, benchmark performance, and run functional localization experiments of visual instruction following.

3.1.1 Question Answering

To gain an understanding of how fine-tuning affects LLaVA, we benchmarked both the pretrained and fine-tuned models on standard text-based and visual (multimodal) question answering (QA) benchmarks (Figure 3.1). Comparing their performance across modalities provides a first indication of how fine-tuning impacts LLaVA’s ability to integrate image understanding into its overall capabilities.

MMLU. The Massive Multitask Language Understanding benchmark (MMLU) [50] is widely used to test factual knowledge across diverse academic and professional fields in state-of-the-art LLMs [51–53]. It is purely text-based, consisting of multiple-choice questions grouped into subjects (e.g., history, physics, law). Each question provides only a stem and answer options, with no additional context. Although MMLU contains no images, it serves as a text-only baseline to anchor our results to widely reported LLM baselines. In this work, we used the Hugging Face implementation [cais/mmlu](#) [54].

ScienceQA. The ScienceQA dataset [55] extends QA into the multimodal domain and is one of the benchmarks used to train and evaluate LLaVA models [5, 6, 56, 57]. Each question includes a stem and answer choices, along with optional supporting context (text and/or an image). Unlike MMLU, which never provides context, ScienceQA includes it. Importantly, if a ScienceQA item contains an image, there is no paired version where the same question is provided with a textual description of the image. For this study, we used the Hugging Face implementation [yerevann/coco-karpathy](#) from Huggingface [58].

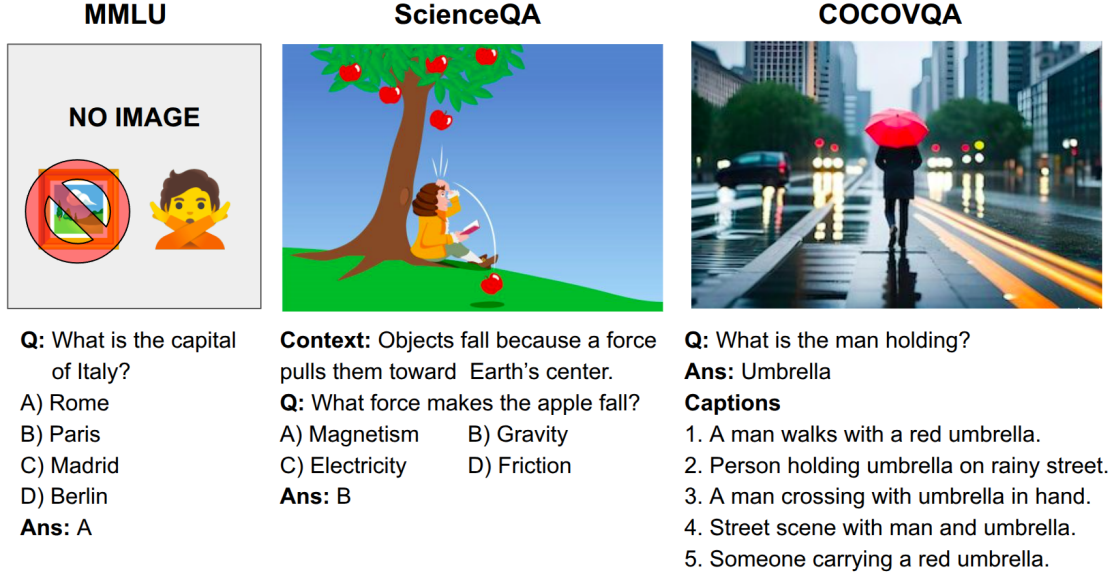


Figure 3.1: Benchmarks used in this study. (1) MMLU—text-only, multiple-choice QA; (2) ScienceQA—multiple-choice QA with optional text and/or image context; (3) COCOVQA—open-ended (single-word) QA built from COCO/COCOQA, with matched image and caption variants for the same items.

Custom Dataset: COCOVQA. While ScienceQA allows comparison across modalities, its multimodal questions do not have textual equivalents with identical content. To enable direct comparison on the same task and content but different modalities, we constructed a custom benchmark by combining COCO [34] and COCOQA [31]. COCO contains images with multiple captions but no questions, whereas COCOQA provides images (sourced from COCO) with questions and single-word answers but no captions.

Since COCOQA questions are linked to COCO images, we merged both datasets to create a unified set where each image is paired with captions, questions, and single-word answers. This allows evaluation under equivalent conditions in both text-based and visual QA. Unlike ScienceQA, here any given question can be answered either by looking at the image or by reading the caption, the latter often being easier for language models. This makes COCOVQA a more rigorous benchmark for comparing performance across modalities.

Specifically, we used the “rest of validation set” (`restval`) split of COCO from the Hugging Face dataset repository [yerevann/coco-karpathy](#)—that implements the Karpathy splits for COCO [59]—and the “test” set of COCOQA from the Hugging Face dataset repository [ThucPD/coco-qa-vi](#) [60]. To avoid multiple QA samples per image during evaluation, we randomly selected one question-answer pair for each image. We refer to this combined dataset as COCOVQA, to distinguish it from COCOQA.

Evaluation Metric. Following the response format prompt for evaluation in [6] we prompted the models to *answer the question with a single word or phrase* and formatted the prompt using the chat template of the model. Since the pretrained and fine-tuned models are chat models this makes the inputs to the models match the statistical patterns learned during fine-tuning for next-token prediction and thus makes the models evaluation more robust [61, 62]. Testing without this formatting degraded performance, particularly the pretrained model, confirming the importance of using response-format prompts aligned with training distributions.

All the question answering tasks used zero-shot evaluation. The benchmarks evaluated here are multiple-choice questions, where the answers are one character in the case of MMLU and ScienceQA (A, B, C, D, E) or open-ended questions, where the answers are a single word in a finite set of words, which is the case of COCOVQA. The metric for these kind of datasets in the literature is exact match accuracy, either directly for multiple-choice questions or after a preprocessing step for open-ended questions [6, 30, 31].

For the multiple-choice questions we made the models generate a single token and for the open-ended questions a maximum of 10 tokens. The latter is a trade-off between not letting the model generate too much text that could end up getting it right by chance and the fact that the model should be able to generate a single word or phrase as instructed. We used up to $N = 2500$ samples for evaluation purposes for reasons explained in Section 3.3.1.

3.1.2 Image Captioning

For image captioning we use the COCOVQA images and instruct the model to produce a single-sentence caption with the following prompt:

```
"Please look carefully at this image and provide a short, descriptive caption. "  
"Focus on the main objects, actions, and scene elements. "  
"Keep the caption concise but informative, similar to how you would describe "  
"the image to someone who cannot see it. "  
"Be specific about what you see without being overly detailed. "  
"THE CAPTION SHOULD BE A SINGLE SENTENCE, NOT MULTIPLE SENTENCES!!!"
```

The instruction is wrapped with the model’s chat template for consistency with training-time input format. We set the maximum generation length to 100 tokens to verify instruction-following (single-sentence constraint) without truncating valid outputs.

Evaluation Metrics

We evaluated generated captions with two complementary metrics:

CIDEr. The Consensus-based Image Description Evaluation (CIDEr) metric [63] measures how closely a machine-generated caption matches what most humans would say about the image, i.e., its “consensus”. It converts both the candidate caption and multiple human-provided references into vectors of n-grams, weighting each n-gram by Term Frequency-Inverse Document Frequency (TF-IDF) to downplay too-common words and highlight informative ones. It then computes cosine similarity between the generated

and reference vectors, effectively rewarding captions that echo the human consensus [64]. CIDEr has been shown to correlate strongly with human judgment in image captioning tasks, particularly when multiple reference captions are available, making it a standard benchmark for evaluating image captioning systems.

CLIP-Score. The CLIP-Score [65] is a reference-free evaluation metric for image captioning that leverages the multimodal CLIP model [7] to measure the semantic alignment between a caption and its corresponding image. By encoding both the image and generated caption into a shared multimodal embedding space and computing their cosine similarity, the CLIP-Score quantifies how well the caption captures the image content at a high conceptual level, prioritizing visual-semantic consistency rather than surface-level text similarity [65]. This approach is especially useful when reference captions are unavailable or insufficient, making CLIP-Score a principled and widely adopted tool for evaluating the fidelity of image-to-text generation tasks.

3.2 Residual Stream Extraction Method

Representational similarity analyses of neural networks such as transformer models requires extracting the residual stream representations of a dataset. Figure 3.2 shows a LLaMA-style decoder block and the tap points used for residual stream extraction in this work. To make these tap points precise, we first review how attention is implemented in transformers.

In transformer models, attention is implemented as multi-head attention [9]:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O, \quad (1)$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$ are the per-head projections, and $W^O \in \mathbb{R}^{Hd_k \times d}$ is the output projection matrix. Here H is the number of heads and $d = Hd_k$ is the model dimension. In a decoder block, masked self-attention is used to ensure causal language modeling, so $Q = K = V = x$ (after normalization).

The contribution of each head to the residual stream is:

$$\text{head_proj}_i = \text{head}_i W_i^O \quad (2)$$

where $\text{head}_i \in \mathbb{R}^{T \times d_k}$, $W_i^O \in \mathbb{R}^{d_k \times d}$, hence $\text{head_proj}_i \in \mathbb{R}^{T \times d}$.

In our experiments, for every layer of the pretrained and fine-tuned LLaVA’s LLM-backbones we recorded (Figure 3.2):

1. the per-head attention output projection head_proj_i (Eq. 2), before the residual connection; and
2. the output of the decoder layer.

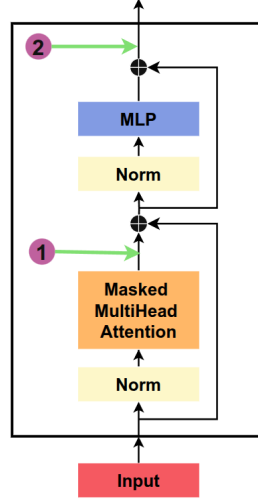


Figure 3.2: LLaMA-style pre-norm decoder block and the two tap points for the residual stream extraction: (1) per-head attention projection after W^O (pre-residual connection), (2) output of the decoder layer.

In decoder-only transformers, the last token in the sequence is the most contextualized and, in the final layer, feeds the next-token logits. Following prior work on transformers interpretability [44–46], we use the last-token embedding as the sequence’s representative in most experiments. Thus, for each input, a single d -dimensional vector is extracted at each tap point for analysis. Unless stated otherwise, all analyses in this work use the residual stream at each layer output.

For implementation, we used Hugging Face checkpoints of [LLaVA-v1.5-7B](#) [6] and [Vicuna-v1.5-7B](#) [66]. To instantiate the pretrained LLaVA setting, we pair the Vicuna-v1.5-7B backbone with the released [pretrained MLP connector](#) [6] and kept the rest of the model architecture unchanged (see Figure 1.1; Section 2.3); the fine-tuned LLaVA is the public instruction-tuned checkpoint.

3.3 Representational Similarity Measures

To examine how fine-tuning alters the internal geometry of LLaVA, we compared the pretrained and fine-tuned models using established measures of representational similarity. Given two sets of representations $X \in \mathbb{R}^{N \times d}$ and $Y \in \mathbb{R}^{N \times d'}$ extracted from the same dataset, these methods quantify the degree to which the two sets encode similar information [40]. In our case, X and Y correspond to the residual stream embeddings of the last token across inputs, obtained from different LLaVA checkpoints.

We focus on two methods (Neighborhood Overlap and Linear CKA) that have proven effective for analyzing representation similarity in deep neural networks, particularly transformers [41, 42, 67].

3.3.1 Neighborhood Overlap

Neighborhood Overlap (NO) measures how much of the local geometry of the representation space is preserved between two sets of embeddings through the nearest neighbors of each data point [40, 41]. For each sample, it compares the k -nearest neighbors—measured with respect to a distance metric—in X with those in Y and computes the fraction of overlap:

$$\text{NO}(X, Y) = \frac{1}{Nk} \sum_{i=1}^N |\mathcal{N}_k(X_i) \cap \mathcal{N}_k(Y_i)|$$

where $\mathcal{N}_k(X_i)$ denotes the set of k nearest neighbors of sample $X_i \in \mathbb{R}^d$. By definition, $\text{NO}(X, Y) \in [0, 1]$, with higher values indicating stronger preservation of local structure between the two representation spaces.

In our experiments, we used the Euclidean distance as the distance metric and set $k = 30$, following prior work showing this value sufficiently captures structural similarity in neural networks in a similar setting [41, 45, 46]. To balance accuracy and computational cost, we fixed the maximum number of samples to $N = 2500$, consistent with the scale used in earlier studies.

3.3.2 Linear CKA

Linear Centered Kernel Alignment (CKA) [40, 42, 68] is a measure of the degree of linear correlation between two representation spaces, which is invariant to isotropic scaling and orthogonal transformations. It is a normalized version of the Hilbert-Schmidt Independence Criterion (HSIC) [69], which relies on the kernel trick to detect non-linear dependence between two variables, and was introduced to the deep learning context by [42]. It has been shown to perform comparably to RBF-based CKA while being computationally efficient as it directly uses feature-space dot products. It is computed as follows:

$$\text{CKA}(X, Y) = \frac{\|Y^\top X\|_F^2}{\|X^\top X\|_F \cdot \|Y^\top Y\|_F}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. We can clearly see that it quantifies the linear correlation between the two spaces by summing the squared covariances between every pair of features across the two spaces. Due to the normalization of the self-similarities the influence of scaling is removed and $\text{CKA}(X, Y) \in [0, 1]$.

3.4 Geometry of Representations

3.4.1 Intrinsic Dimension of Representations

To complement similarity-based analyses, we also examined how fine-tuning changes the geometric complexity of LLaVA’s internal representations by estimating their intrinsic dimension (ID). Whereas similarity measures capture overlap between pretrained

and fine-tuned checkpoints, ID provides a different perspective: it quantifies whether fine-tuning makes the representation manifold more compact, more redundant, or more expressive.

Formally, the intrinsic dimension of a dataset is the minimum number of degrees of freedom required to locally describe its data manifold [43]. In this work, the ID of each representation matrix $X \in \mathbb{R}^{N \times d}$ was estimated using the Generalized Ratios Intrinsic Dimension Estimator (GrIde) [70], an extension of the two-nearest-neighbor (2NN) method [71]. GrIde builds on the result that, when data points lie on a smooth manifold of dimension δ with constant density, the ratio between distances to successive neighbors follows a distribution that depends only on δ [70, 71].

For a given point $X_i \in \mathbb{R}^d$, GrIde generalizes the computation of the intrinsic dimension to arbitrary neighbor ranks k_1 and k_2 , with distances r_{i,k_1} and r_{i,k_2} , with respect to a distance metric. In the original 2NN estimator [71], these ranks were fixed at $k_1 = 1$ and $k_2 = 2$. A common choice in GrIde [45, 70] is $k_1 = k$ and $k_2 = 2k$, where k is the base neighbor rank. This generalization enables multi-scale analysis and reduces bias that may arise at very small scales. The distance ratio $\mu_k = \frac{r_{i,k_2}}{r_{i,k_1}}$ is then used to fit a maximum likelihood model of the form [70]:

$$\mathcal{L}(\delta) \propto \delta(\mu_k^\delta - 1)^{k-1} \mu_k^{(1-2k)\delta-1}, \mu_k > 1$$

whose maximizer yields the ID estimate at the chosen neighbor scale.

Although the method assumes constant density, it has been shown to remain robust as long as density is approximately constant over the relevant neighborhood [70]. In this work, we followed prior studies [41, 45, 46] and used $k = 16$, corresponding to neighbor ranks $k_1 = 16$ and $k_2 = 32$, with Euclidean distance as the metric. All computations were performed using the [Dadapy](#) library [72].

3.4.2 Modality Gap

Experiment Setup. To evaluate how fine-tuning reshapes LLaVA’s backbone representations, we examined the *modality gap*—the geometric separation between text and image embeddings. This analysis indicates whether fine-tuning promotes tighter integration between modalities or preserves their separation.

Unlike CLIP—which uses separate encoders and returns individual global embeddings for text and for images—LLaVA is a decoder model that encodes both modalities in a shared representational space and does not yield a single, fixed embedding per modality. Therefore, representative embeddings for each modality must be constructed in a task-specific way. Using solely the last embedding in a sequence as in Section 3.2 might not be suitable to fully represent textual and visual information from the inputs because although this embedding is the most contextualized, it also contains the information for the next token prediction, which might bias the representation.

In this work, we extracted text and image embeddings from image QA and image captioning tasks using the COCOVQA dataset. The core idea is to control the order of text and image input content so that the modality placed last is the most contextualized;

we then select a representative from those embeddings. Concretely, we *randomly choose one embedding per sample*, which might mitigate next-token bias while preserving the modality’s contextual information.

By default, LLaVA’s chat template places image embeddings before text embeddings. For QA, the structure of the prompt is:

```
<SYSTEM MESSAGE>
USER: <IMAGE>
<QUESTION>
Answer the question with a single word or phrase.
ASSISTANT: Answer:
```

where <SYSTEM MESSAGE>, <IMAGE>, and <QUESTION> are placeholders for the system message, image, and question about the image. Here, <IMAGE> is a special token replaced by image embeddings after tokenization. The only thing the chat template does is to structure the multiple input contents (e.g., system message, image, and question in this case) into a single string that forms the conversation between the **USER** and **ASSISTANT**, ensuring compatibility with the model’s instruction-following behavior.

In this setup, text tokens appear at the end and are therefore the most contextualized. For each sample, we randomly select one text embedding to represent the textual modality, focusing in the embeddings of the question.

To make image embeddings maximally contextualized, we manually reversed the input order so that text comes before the image, as in:

```
<SYSTEM MESSAGE>
USER: <QUESTION> <IMAGE>
Answer the question with a single word or phrase.
ASSISTANT: Answer:
```

In this case, we randomly select one of the image embeddings for each sample.

A similar setup was used for image captioning. To enrich the context, we provided the model with the ground-truth captions and asked it to generate an additional description. Unlike the captioning setup in Section 3.1.2, here the task is to extend existing captions rather than generate one from scratch. The prompt looks like:

```
<SYSTEM MESSAGE>
USER: <IMAGE>
Below are 5 numbered descriptions of the image.
Please generate an additional description of the image.
The caption should be a single sentence, not multiple sentences!!!
```

1. <CAPTION 1>
2. <CAPTION 2>
3. <CAPTION 3>

4. <CAPTION 4>
 5. <CAPTION 5>

ASSISTANT: Answer:

From these tasks, we obtained two matrices, $X_{\text{text}}, X_{\text{image}} \in \mathbb{R}^{N \times d}$, where N is the number of samples and d the representation dimension. We then computed two measures:

1. Median cosine similarity (input-wise): captures the directional alignment of text vs. image embeddings in high-dimensional space. Importantly, embeddings may align directionally yet still form separate modality-specific clusters [47, 48].
2. Homogeneity score after clustering: using the Advanced Density Peaks (ADP) clustering algorithm [73], we evaluated whether embeddings form distinct clusters by modality. The homogeneity score [74] quantifies this separation.

Together, these measures provide complementary perspectives: cosine similarity captures cross-modal alignment, while clustering-based homogeneity reveals whether text and image embeddings remain segregated in representational space. This follows the methodology of Serra et al. [46], who examined modality gaps across different VLMs, including LLaVA. Together, these measures allow us to test whether visual instruction tuning reduces the modality gap or preserves their separation, thus providing concrete evidence of how fine-tuning reshapes the internal geometry of LLaVA’s representations.

Because cosine similarity is a widely known metric, here it is not described further; instead, the Advanced Density Peaks (ADP) clustering algorithm, which underlies the cluster structure used in this analysis, is described next, followed by the homogeneity score that quantifies cluster purity. Together, these sections connect the experimental setup with the clustering-based evaluation of the modality gap.

Advanced Density Peaks (ADP). ADP clustering [73] is a non-parametric density-based clustering algorithm that automatically determines the optimal number of clusters by identifying statistically significant density peaks in high-dimensional datasets. The algorithm operates on the fundamental principle that cluster centers correspond to local density maxima in the data manifold, while clusters are naturally separated by regions of lower density.

The method follows a three-stage process. First, it estimates the local density ρ_i of each data point X_i using a k-nearest neighbor approach, computing both density values and their statistical uncertainties ε_i . The local densities $\rho_{i,k} = \frac{k}{NV(r_k, \delta)}$ are estimated on the intrinsic manifold, where $V(r_k, \delta)$ is the volume of the k -dimensional sphere of radius r_k in a d -dimensional space with intrinsic dimension δ . The algorithm then identifies putative cluster centers as points that represent local density maxima within their k-nearest neighbor regions. Each remaining point is then preliminarily assigned to its nearest density peak based on density gradients toward higher-density regions. Finally, for each peak, the algorithm considers the saddle point shared with another

peak, with density $\rho_{c,c'}$ and uncertainty $\varepsilon_{c,c'}$. It computes the standardized density drop test statistic:

$$T = \frac{\log \rho_c - \log \rho_{c,c'}}{\varepsilon_c + \varepsilon_{c,c'}},$$

where ρ_c and ε_c are the density of the peak of the cluster c and uncertainty of the peak density, respectively. If this value does not exceed a predefined Z-score confidence threshold, the two peaks are merged. This merging step is applied iteratively until only clusters supported by statistically significant peaks remain.

In this work, the local density of each point is computed with a fixed k -nearest neighbor estimator ($k = 16$), following established practices in representation analysis [41, 45, 46] as it has been shown to work better for representations of neural networks, rather than using point-adaptive density estimation methods [73]. The intrinsic dimension of the data representation is first estimated with the Grice algorithm (Section 3.4.1) using the Cosine distance, and this value is used in the density computation. The confidence threshold for merging peaks was set to $Z = 1.65$, corresponding to 95% one-tailed confidence in a Z -test. For clustering the image and text embeddings together their extracted representation matrices are concatenated into a $2N \times d$ matrix. All ADP clustering steps are performed using the `Dadapy` library. The algorithm returns the cluster labels for each point.

The clustering performance is evaluated using the homogeneity score, a clustering validation metric that measures whether each cluster contains only members of a single class [74]. The homogeneity score is computed as $h = 1 - H(C|K)/H(C)$, where $H(C|K)$ represents the conditional entropy of class distribution C given the cluster assignment K and $H(C)$ is the entropy of the overall class distribution. This metric ranges from 0 to 1, where a score of 1 indicates perfect homogeneity (each cluster contains samples from exactly one class, so knowing the cluster assignment completely determines the class label and leaves no uncertainty about it) and 0 represents the worst case scenario where class distributions within clusters are completely mixed, so the cluster assignment gives no information about the class.

3.5 Functional Localization via Layer Patching

We test whether the capabilities gained through visual instruction tuning are *localized* to specific backbone layers or *distributed* across the network. To do so, we conducted *layer patching* experiments: layers from the fine-tuned LLaVA backbone are inserted into the pretrained backbone either (i) cumulatively from the input to the output of the network, or (ii) in a sliding window fashion that replaces only a contiguous subset at a time. Performance differences across tasks indicate where the fine-tuned functionalities emerge. The tasks used for these experiments are question answering and image captioning.

Only the subset of LLM backbone layers being tested changes between runs; the vision encoder and connector remain fixed, and the LM head is kept unless explicitly stated.

Question Answering. We evaluate exclusively on the COCOVQA dataset to enable controlled, like-for-like comparisons across modalities with identical content. For each configuration (cumulative or sliding-window), we replace a specified set of pretrained backbone layers with their fine-tuned counterparts and measure zero-shot accuracy under the same prompting and formatting used in Section 3.1.1.

The rationale is that by holding the task and input distribution fixed while varying which layers are patched, we isolate which layers in the fine-tuned backbone contribute the most to multimodal QA, and whether improvements accumulate across layers or hinge on a few critical ones.

Image Captioning. The image captioning setup is the same as in Section 3.1.2.

3.6 Computational Resources and Reproducibility

All our experiments ran on a single NVIDIA A100 (40 GB) GPU on the ORFEO cluster of AREA Science Park. To promote determinism and reproducibility, we fixed the random seed to 42 and disabled PyTorch stochasticity. For all model generations we used greedy decoding, which in Huggingface is implemented as `generate(..., do_sample = False)`. All models were loaded in `fp16` (half-precision).

Chapter 4

Results and Discussion

In line with the thesis objective—to *characterize how visual instruction tuning reshapes the LLaVA backbone and how the representational changes affect multimodal performance*—this chapter presents the experimental results. Each section corresponds to one methodological block from Chapter 3 and links the findings directly to the research questions introduced in Chapter 1.

The chapter begins with a broad view of models’ performance and representation similarity at different levels of the residual stream. It then moves to a finer-grained analysis of the geometry of data representations. Finally, a functional inspection of the LLaVA backbone through layer patching validates and contextualizes the earlier results.

4.1 Zero-shot QA: Performance Effects of Visual Instruction Tuning

Before examining internal changes in the LLaVA backbone, we first quantify the performance impact of visual instruction tuning across text and multimodal QA tasks. We compare zero-shot accuracy of the fine-tuned checkpoint to the pretrained one—which is LLaVA with just feature alignment and Vicuna v1.5 7B as the LLM backbone (see Sections 2.3 and 3.2)—on the benchmarks described in Section 3.1.1.

Table 4.1 summarizes the results. The fine-tuned model outperforms the pretrained model on every dataset. The largest gain is on COCOVQA, especially on the multimodal subset, where accuracy rises by $\sim 58\%$ relative to the pretrained model (41.88%). On COCOVQA, both checkpoints score higher on text than on multimodal (58.36% vs. 41.88%); the fine-tuned model roughly halves this difference (from 16.75% to 8.04%). Coincidentally, on ScienceQA, the fine-tuned model achieves the same accuracy on the text and multimodal subsets (both 66.53%).

Prompting and format sensitivity. Careful prompt formatting—e.g., instructing the model to “*answer with a single word or phrase*”—improves zero-shot performance for both checkpoints, as noted in the LLaVA reports [6] and prompt-engineering studies [62,

Benchmark	Modality	Pretrained LLaVA	LLaVA
MMLU	Text	48.67%	50.63%
ScienceQA	Text	64.25%	66.53%
ScienceQA	Multimodal	58.11%	66.53%
COCOVQA	Text	58.36%	74.56%
COCOVQA	Multimodal	41.88%	66.52%

Table 4.1: Zero-shot accuracy (%) of the pretrained and fine-tuned LLaVA (v1.5 7B) on MMLU, ScienceQA (text and multimodal), and COCOVQA (text and multimodal).

75]. On MMLU, aligning the prompt with training-time formats helps reconcile differences with prior reports for Vicuna (LLaVA pretrained LLM backbone) and LLaMA-2 (Vicuna pretrained LLM backbone) [45, 51, 66], which can vary with few-shot setup and inference instructions. We explored alternative prompt formats and the fine-tuned model is more robust and maintains higher zero-shot accuracy across these variants. Beyond multimodal inputs, visual instruction tuning also benefits purely textual QA, consistent with LLaVA’s rich instruction data (descriptions, dialogue structure, reasoning steps; see Section 2.3). Our ScienceQA multimodal accuracy (66.53%) closely matches LLaVA-1.5 7B reports (66.8%) [76]. On COCOVQA, the fine-tuned model exceeds classic pre-Transformer VQA systems [30, 31], whereas the pretrained checkpoint does not.

Task type matters. Gains are modest on MMLU and ScienceQA but substantial on COCOVQA. A likely reason is the *match* between training objectives and evaluation format: LLaMA-2/Vicuna training and evaluation emphasize multiple-choice datasets [51, 66] (PIQA, SIQA, HellaSwag, WinoGrande, OBQA, CSQA, MMLU), whereas LLaVA’s instruction tuning includes open-ended VQA and caption-style supervision [5, 6] (VQAv2, GQA, OKVQA, OCR-VQA). Since the LLaVA backbone inherits from LLaMA-2 and Vicuna, which were trained on multiple-choice datasets, improvements on such datasets remain modest. By contrast, LLaVA’s fine-tuning on open-ended tasks yields much larger gains. Consistent with this, the pretrained model (a LLaMA-2 descendant) typically scores lower on open-ended QA than on multiple-choice, and the largest fine-tuning deltas appear on the open-ended COCOVQA subsets.

These behavioral patterns motivate the representational analyses that follow: *where in the backbone do these fine-tuning gains arise, how are text and image information integrated with depth, and how are they reflected in the internal geometry of the model?*

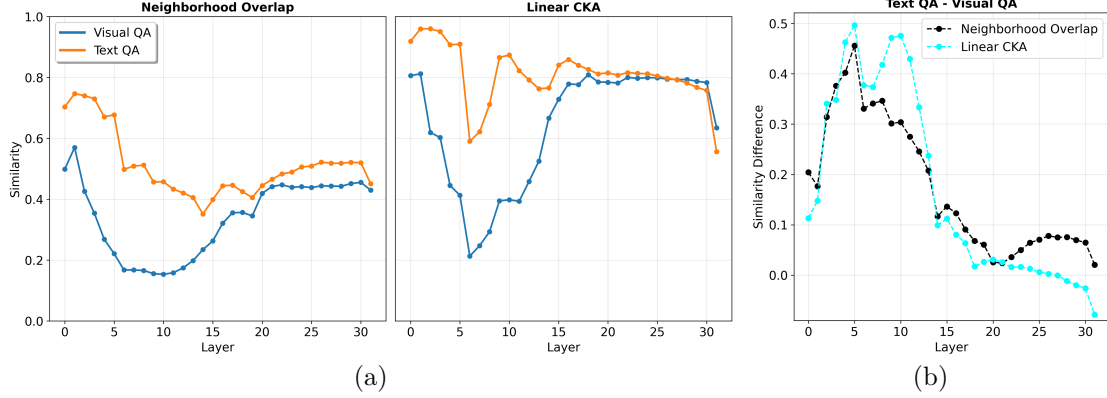


Figure 4.1: Layer-wise similarity between fine-tuned and pretrained LLaVA. (a) NO (left) and Linear CKA (right) text (orange) and visual (blue) QA on COCOVQA. Higher values indicate greater cross-checkpoint similarity. (b) Text-multimodal difference (black: NO, cyan: linear CKA); positive values mean higher similarity on text than on multimodal representations.

4.2 Where the Backbone Changes: Representational Similarity

We compare residual-stream representations of the fine-tuned and pretrained LLaVA at two levels of granularity: (i) *layer outputs* and (ii) *per-head attention projections* (Section 3.2). Similarities are measured using Neighborhood Overlap (NO) and Linear CKA (see Section 3.3) on embeddings extracted from the COCOVQA dataset in text and visual QA settings (Section 3.1.1). This analysis reveals *where* across depth and *which* heads undergo the largest changes after fine-tuning.

Coarse-grained: similarity at the layers level. Figure 4.1a shows layer-wise residual stream similarity between the fine-tuned and pretrained checkpoints. Both NO and Linear CKA indicate substantial representational change (values well below 1). Across depth, text representations are consistently more similar than multimodal ones. In both metrics, similarity decreases sharply in the early layers, reaching a minimum around layer 7 for multimodal data, before gradually recovering toward the deeper layers. Similar trends were observed between the text and visual QA representations on ScienceQA.

The difference curves in Figure 4.1b, showing the difference between text and multimodal similarity, *though not standard*, highlights this contrast more directly and makes the underlying pattern easier to compare. The difference grows in early layers, peaking near layer 6 at ~ 0.46 - 0.50 (NO/linear CKA), then steadily shrinks; by layer 21 the NO difference is near zero. In the final layers, Linear CKA becomes slightly negative, indicating that under this metric, multimodal representations across checkpoints are marginally more similar than their text representations.

Fine-grained: similarity at the heads level. While layer-wise similarity localizes the impact of fine-tuning by depth. To resolve where changes occur within a layer, we analyze the *head-wise* residual stream similarity between the pretrained and fine-tuned models, quantifying how effects distribute across heads and whether they concentrate in specific heads.

Figure 4.2a reports head-wise NO at the attention output projected onto the residual stream (see Section 3.2). The layer-wise pattern repeats at head resolution: similarity on text exceeds multimodal, especially in early layers. Compared to text, fewer heads exhibit high similarity on multimodal data; this is clearest in the first two layers, where the text maximum (0.91) is surrounded by heads whose values are close to the multimodal maximum (0.79). Notably the maximum for both modalities occurs at layer 1, head 30. Additionally, some heads show comparable similarity across modalities (e.g., around head 27 in early layers). Linear CKA yielded consistent results.

The difference matrix in Figure 4.2b (head-wise analogue of Figure 4.1b) shows sparse heads where multimodal similarity exceeds text by up to ~ 0.2 (negative values). In early layers (3-11), many heads are more similar on text than on multimodal data (up to ~ 0.5). The mean across heads per layer, and its text-multimodal difference (Figure A.1), aligns with the layer-wise curves. Similar results were observed between the text and visual QA representations on ScienceQA.

Impact of visual instruction tuning on representation similarity. Figures 4.1, 4.2 and A.1 jointly indicate that visual instruction tuning alters the backbone primarily in the early-mid layers. The sharper similarity drop for multimodal inputs in this region suggests that the effect of visual instruction tuning on the LLM-backbone of LLaVA is localized in the first half of the network. By contrast, the small text-multimodal differences in deep layers (Figures 4.1b and A.1b) indicate that late layers mainly coordinate information for next-token prediction, especially on QA. Because the analyzed representations are last-token embeddings and the datasets contain equivalent content in different modalities, these results hint that the initial layers are responsible for multimodal integration. The modest differences in the first two layers may reflect a shared role (e.g., question parsing) prior to strong modality-specific processing. These results contrast with prior work on LLMs [45] where the representations in the late layers changed the most due to fine-tuning.

Fine-tuning updates the model’s weights and thus its internal representations. The reduction in similarity on text is consistent with the fine-tuned model’s stronger instruction-following (Table 4.1).

The lower similarity on multimodal data is expected since visual instruction tuning enables the model to understand images.

The rise in the text-multimodal difference (Figures 4.1b and A.1b) in the early layers indicates that text retains greater stability than multimodal features (text representations get more and more similar than the multimodal representations). This supports the hypothesis that in the initial layers the model progressively incorporates visual information.

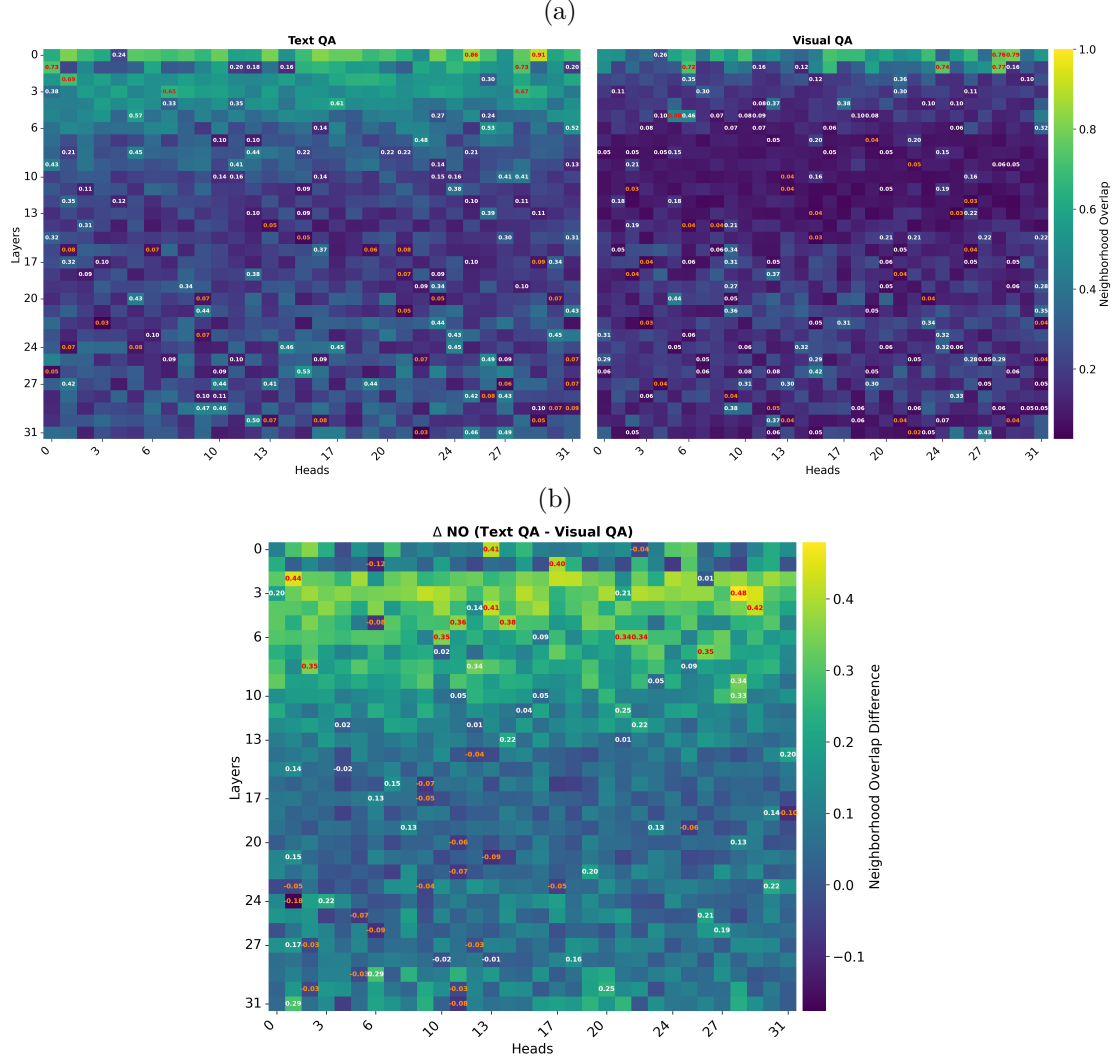


Figure 4.2: Head-wise NO between fine-tuned and pretrained LLaVA. Annotations mark extreme values (white: top/bottom 5% within layer; red: global top 5%; orange: global bottom 5%). (a) Text (left) and visual (right) QA on COCOVQA. (b) Text-multimodal difference (negative: higher multimodal similarity; positive: higher text similarity).

Head-wise results (Figure 4.2) suggest the presence of *modality-agnostic* heads, where similarity values are comparable across modalities. These may correspond to heads that serve similar functions regardless of modality, such as interpreting the question or generating the answer. In contrast, heads with the largest positive differences may indicate strong specialization in multimodal understanding, whereas those with the largest negative differences may reflect enhanced text-specific processing. The head-wise modality differences (Figure 4.2b) show that visual instruction tuning affects all heads, with its strongest concentration in the early layers—though not in tightly clustered groups—consistent with the layer-wise analysis.

4.3 Geometry After Fine-Tuning: Intrinsic Dimension

Having localized where representations change via similarity, this section investigates whether fine-tuning also alters the data manifold geometry of those representations. We estimate the intrinsic dimension for embeddings extracted from COCOVQA under text and visual QA (setup as in Section 3.1.1) and image captioning (prompt as in Section 3.4.2). This tests whether the early-mid layer differences seen in similarity correspond to a systematic geometric reorganization, directly addressing the thesis goal of identifying *where and how* visual instruction tuning modifies the LLM backbone.

Figure 4.3a shows that, for the pretrained and fine-tuned models, ID starts at about 13-17 in shallow layers and then decreases with depth. In the fine-tuned model, text QA maintains consistently higher IDs than multimodal tasks. With the exception of small fluctuations in text-only representations, all tasks peak early (layer~4; ~16-18), dip near layer 10, rise again mid-network, and then decline steadily after layer~20, stabilizing around 9-10 in the deepest layers, where curves for all tasks converge.

In the pretrained model, the second half of the network follows a similar downward trend, but early layers differ: multimodal IDs increase steadily while IDs for text QA and image captioning either fluctuate or grow more slowly. Peaks occur later (layers~13-16), in contrast to the early peak in the fine-tuned model, with visual QA reaching ~21 above the maxima of the other tasks. By the final layers, IDs remain larger overall than in the fine-tuned model, and the three tasks are still separated by ~3 units.

The difference panel in Figure 4.3b makes the effect of fine-tuning explicit: IDs are generally *lower* after fine-tuning, especially in mid layers for multimodal inputs. For visual QA, the gap peaks near layer~12 where the ID difference is ~8, then decreases and rises at a slower rate before stabilizing. Beyond layer~20, differences settle around 5-6 for text QA, 3-4 for visual QA, and near zero for captioning. Notably, for purely textual inputs, IDs in the pretrained model remain higher than in the fine-tuned model throughout depth, with the largest compression appearing toward the end.

A complementary analysis in Figure A.2 of the appendix uses *dataset entropy* [77], an information-theoretic analogue of ID. It is defined as the Shannon entropy of the normalized eigenvalue spectrum of the Gram matrix built from centered and normalized dataset representations. Higher entropy indicates a more diffuse spread of variance across eigen-directions, corresponding to less compact representations and more diverse

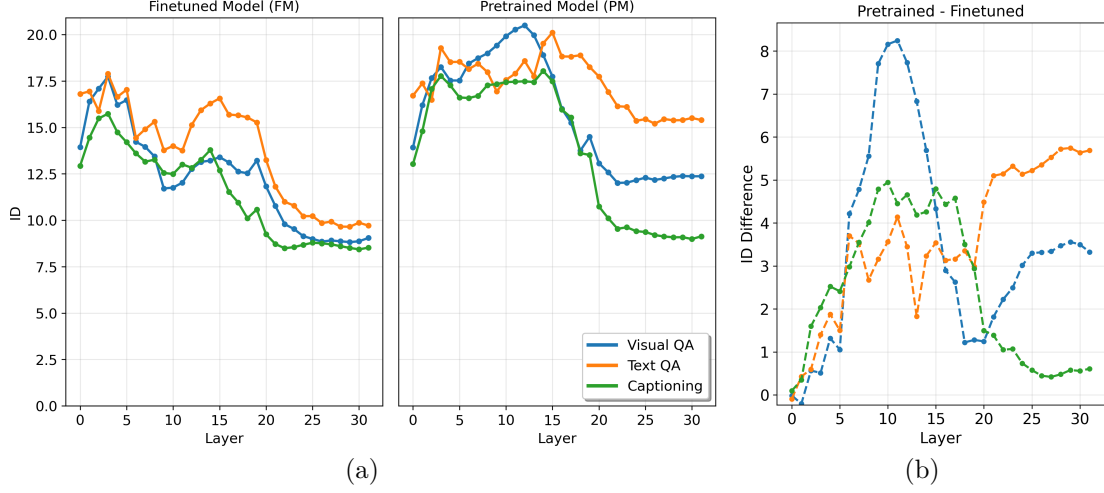


Figure 4.3: Intrinsic dimension analysis for text QA (orange), visual QA (blue), and image captioning (green) on COCOVQA. (a) Finetuned model (left) and pretrained model (right). Higher values indicate higher-dimensional (less compact) representations. (b) ID difference (pretrained – fine-tuned). Positive values mean the pretrained model has higher ID.

features. Its layer-wise pattern broadly mirrors (though not identical) the ID curves: entropy differences (pretrained > fine-tuned) peak around layers 10-12 for multimodal data, then converge in deep layers (text ≈ 0.08 , multimodal ≈ 0.05 , and captioning ≈ 0 , with a residual ≈ 0.04 in the final layer).

What ID reveals about fine-tuning. The change in ID and dataset entropy in Figures 4.3 and A.2 support the hypothesis that the early-mid layers are the ones changing the most in the LLaVA LLM-backbone due to visual instruction tuning, and is consistent with the NO and linear CKA similarity analysis shown in Section 4.2. The compression observed in these layers aligns with prior work on autoregressive transformers using entropy-based measures [77], and the hump-shaped ID profile of the pretrained model matches reports for LLaMA models [45]. Fine-tuning *compresses* the representation manifolds (lower ID/entropy), most strongly for multimodal inputs in mid layers, and with smaller but persistent reductions in late layers (particularly for text). In the deepest layers, IDs across tasks converge toward a common, compact regime, consistent with these layers chiefly coordinating information for next-token prediction. This supports the hypothesis that visual instruction tuning primarily injects and integrates visual grounding in the first half of the backbone, while later layers act as shared decoders across modalities.

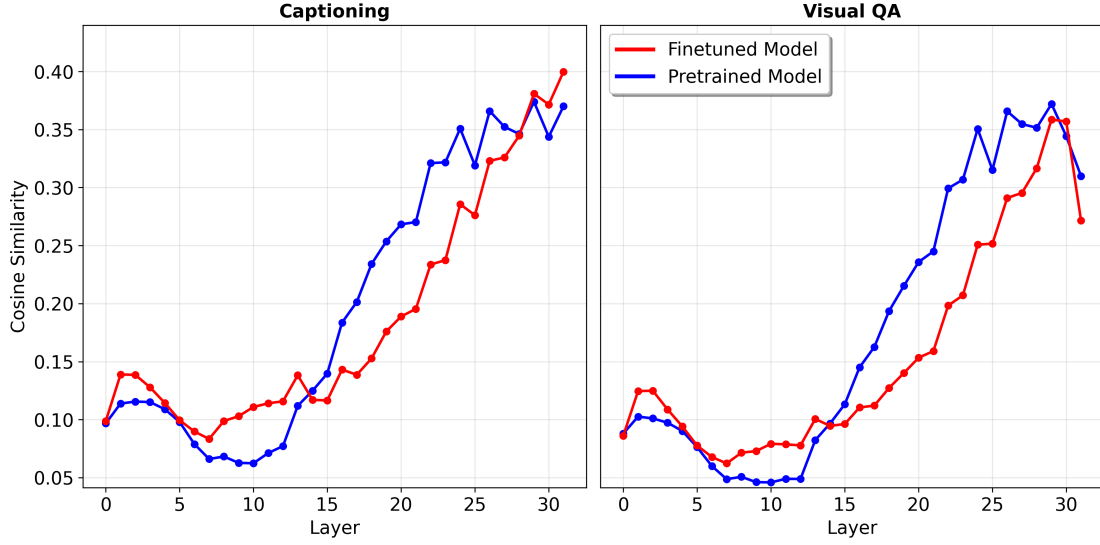


Figure 4.4: Cosine similarity between the most contextualized text and image representations of the fine-tuned (red) and pretrained (blue) LLaVA, for image captioning (left) and visual QA (right) on the multimodal subset of COCOVQA. Higher values indicate stronger directional alignment.

4.4 Modality Gap After Fine-Tuning: Alignment vs. Separation

Having established *where* the backbone changes (via similarity) and *how* geometry is reorganized (via ID) *between the model’s checkpoints*, we now analyze the *relationship between data modalities within models’ checkpoints*. Using the setup in Section 3.4.2, we quantify the text-image *modality gap* with two complementary views: (i) we examine the directional alignment between the most contextualized text and image embeddings by computing the cosine similarity between them, and (ii) analyze the cluster separation and homogeneity with advanced density peak clustering.

Directional alignment. Figure 4.4 shows that, for both captioning (left) and visual QA (right), cosine similarity is initially low in shallow layers, increases from about layer 10, and peaks in the deepest layers. Across most deeper layers, the pretrained model exhibits higher alignment than the fine-tuned model; the fine-tuned model briefly exceeds it in the earliest layers and again at the final layer for captioning.

Cluster homogeneity. Figure 4.5 reports homogeneity after clustering text and image embeddings together with ADP. Scores start near 1.0 (well-separated modality clusters) and then decline with depth, with a sharper drop beginning around layers 10-12, mirroring the rise in cosine similarity. For visual QA (right), the pretrained model maintains

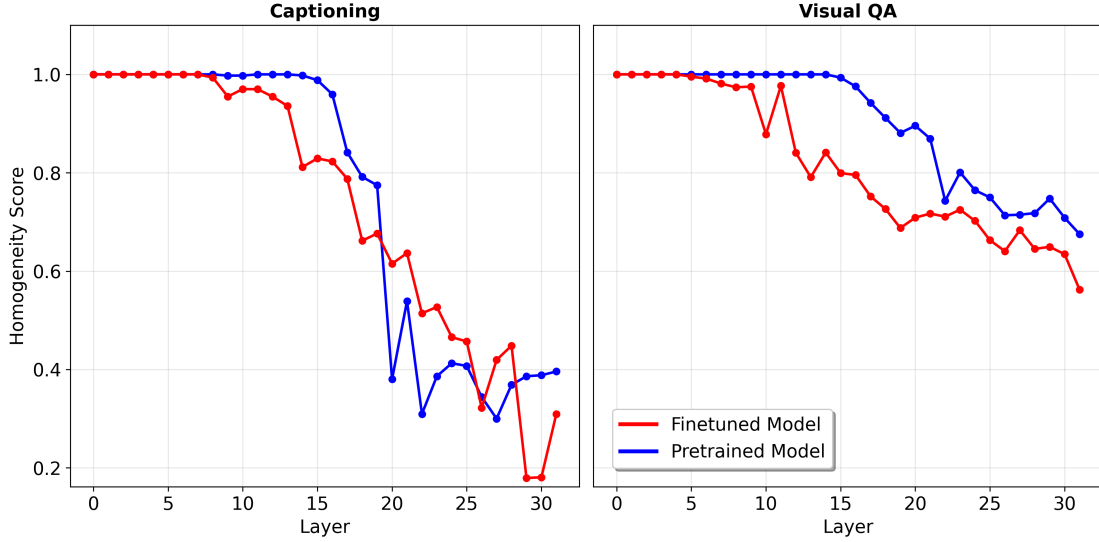


Figure 4.5: Homogeneity after clustering text and image embeddings together (same tasks as in Figure 4.4). A value of 1 indicates perfectly separated modality clusters; lower values indicate stronger mixing.

higher homogeneity than the fine-tuned model across the network, with the largest gap roughly in layers 12-23. For captioning (left), the pretrained curve remains near 1.0 until about layer 15 and then falls more rapidly than the fine-tuned curve; final values are ~ 0.40 (pretrained) vs. ~ 0.30 (fine-tuned), both lower than in QA.

Effect of fine-tuning on the modality gap. The directional alignment (cosine similarity; Figure 4.4) and the clustering structure (homogeneity score; Figure 4.5) follow prior observations for LLaVA’s modality gap [46]. Cosine similarity increases with depth in both pretrained and fine-tuned models, indicating that text and image representations are nearly orthogonal in early layers and become progressively more aligned in mid-to-late layers. Relative to the pretrained model, fine-tuning slightly increases alignment in early layers, whereas in late layers the gap is slightly larger (the pretrained model tends to show higher similarity), consistent with reports that a larger gap can correlate with better downstream performance [47] (see Table 4.1).

The homogeneity score declines with depth (Figure 4.5), meaning that modality-pure clusters (score ≈ 1) in shallow layers become increasingly mixed in deeper layers. On the question-answering task, mid-layer homogeneity is lower for the fine-tuned model than for the pretrained model, suggesting stronger mixing, and thus a smaller gap, in that region. In the final layers, the two tasks diverge and do not yield a single, task-independent conclusion.

Summary and scope. Directional alignment (cosine similarity) and cluster homogeneity do not exhibit a single, consistent pattern across tasks. In early layers, fine-tuning

slightly increases text-image alignment while modality clusters remain well separated. Deeper in the network, alignment is generally lower for the fine-tuned model; however, cluster separability diverges by task-embeddings mix more for QA and slightly less for captioning. These findings are specific to the tasks evaluated, our setting and to the specific fine-tuning setup of LLaVA (one-epoch update of the LLM backbone).

4.5 Task-Level Validation via Layer Patching

This section applies the patching protocol from Section 3.5 to *text VQA*, *visual VQA*, and *image captioning* (all built from COCOVQA). We form *hybrid* models by replacing subsets of the pretrained backbone with the corresponding fine-tuned layers using two schemes: (i) *cumulative* patching and (ii) *sliding-window* patching. In the cumulative scheme, we patch from the start to the end of the network and denote by ℓ the *last* included layer index (0-indexed), i.e., layers $[0:\ell]$ are taken from the fine-tuned model. In the sliding-window scheme, we patch a contiguous window of size w starting at index ℓ , i.e., layers $[\ell:\ell+w]$ are patched.

4.5.1 Question Answering

Accuracy increases as more fine-tuned layers are *cumulatively* patched (Figure 4.6). On *visual QA*, performance rises with ℓ : 19% above the pretrained reference (0.42) at $\ell = 2$; 85 – 98% of the fine-tuned reference (0.66) for $\ell = 4 – 12$; and from about 12 layers onward, the hybrid matches or exceeds the fine-tuned model. When all backbone layers are patched *except* the final LM head (output layer), accuracy reaches 103% of the fine-tuned model. On *text QA*, accuracy stays at or slightly below the pretrained reference for shallow patches, then increases after 14 layers and eventually matches the fine-tuned model.

For a complementary view, the *sliding-window* results (Appendix; Figure A.3) patch a single layer at a time ($w = 1$) starting at index ℓ . Effects are small and localized: on text QA, accuracies hover near the pretrained reference; on visual QA, early-layer patches yield a modest bump that fades in mid-to-deep layers. No single-layer patching approaches the fine-tuned reference.

4.5.2 Image Captioning

Image captioning shows the same monotonic trend under *cumulative* patching. CIDEr is near zero for the pretrained model and remains small for shallow patches ($\ell = 1 – 7$); it rises sharply for $\ell \approx 9 – 15$ and then climbs more gradually, peaking at 0.81 for $\ell = 31$ (all backbone layers are patched except the LM head), close to the fine-tuned reference (0.82). CLIP-Score follows the same pattern, moving from 0.14 – 0.16 for shallow patches to 0.23 for deep patches, slightly above the fine-tuned reference (0.22) and well above the pretrained reference (0.15).

The complementary *sliding-window* analysis (Appendix; Figure A.4) confirms localization: patching a single layer ($w = 1$) yields small, position-specific effects (CIDEr

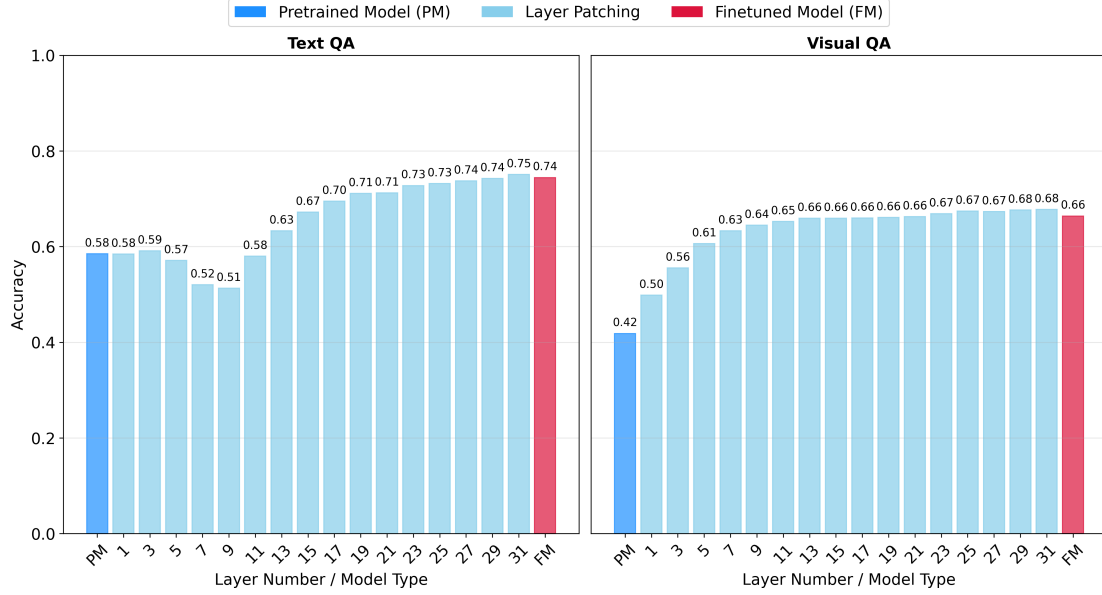


Figure 4.6: COCOVQA question-answering accuracy for hybrids formed by *cumulatively patching* layers $[0:\ell]$ from the fine-tuned backbone into the pretrained backbone (light-blue bars). The x-axis is ℓ . Left: text-only QA. Right: visual QA. Blue and red bars denote the pretrained (PM) and fine-tuned (FM) references, respectively.

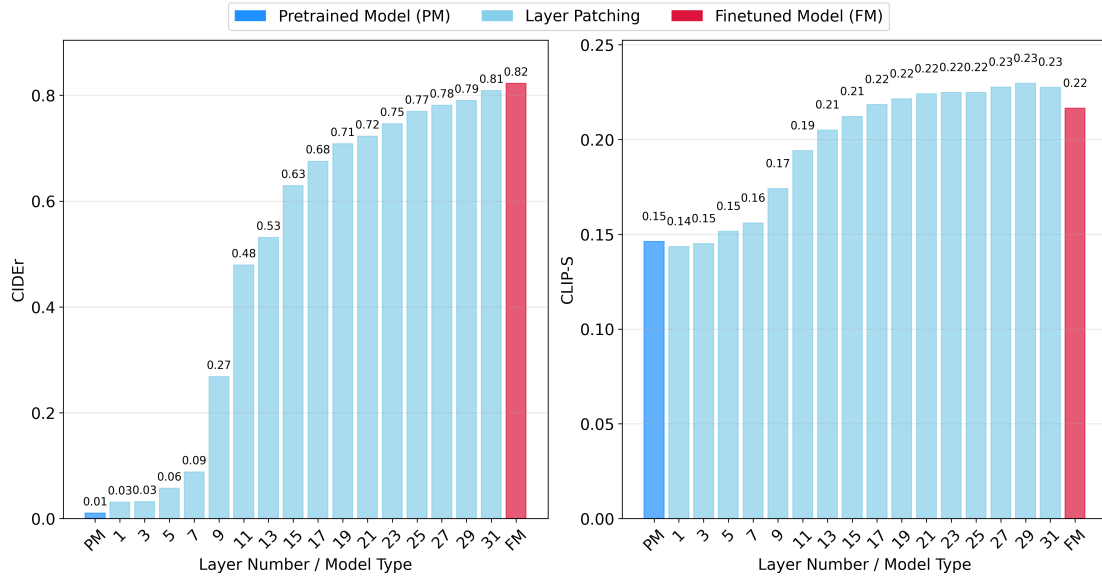


Figure 4.7: Image captioning performance under *cumulative* patching of layers. Similar to Figure 4.6. Left: CIDEr. Right: CLIP-Score.

peaks around ~ 0.10 - 0.11 in early-mid layers; CLIP-Score remains ~ 0.15 - 0.16). No single-layer patch approaches the fine-tuned reference.

What patching reveals. Cumulative hybrids models (pretrained layers combined with fine-tuned ones) exhibit a clear performance increase followed by a plateau on both QA and image captioning (Figures 4.6 and 4.7). The steep gains for early-mid layers indicate that visual instruction tuning primarily edits this region, which carries visual understanding and multimodal integration. Beyond that, returns diminish, suggesting late layers chiefly coordinate/decoding for next-token prediction rather than adding multimodal capability. A modest dip on text QA around $\ell \approx 8 - 10$ (see Figure 4.6) may also be a consequence of fine-tuning, though evidence is limited. The slight performance excess of the cumulatively patched model that includes all backbone layers except the LM head, relative to the fine-tuned checkpoint, is likely attributable to LM-head updates during fine-tuning (the gain is small). The sliding-window results (Appendix; Figures A.3 and A.4) reinforce this picture: improvements concentrate in early-mid layers and are otherwise small. Overall, these task-level outcomes functionally validate the representational findings: instruction tuning embeds visual knowledge mainly in early-mid LLM-backbone depth, while late layers behave as shared decoders across modalities.

Chapter 5

Conclusion

5.1 Key findings

This thesis investigated *how* visual instruction tuning alters the internal representations of LLaVA’s LLM backbone and *where* in depth those changes concentrate. We first showed that visual instruction tuning improves zero-shot QA performance, with the largest gains on open-ended, multimodal inputs. To localize where the backbone changes, we compared residual-stream representations between fine-tuned and pretrained checkpoints using Neighborhood Overlap and linear CKA at both the *layer* and *head* levels. Similarities drop most in early-mid layers—especially for multimodal inputs—with partial convergence across modalities in deeper layers for QA (Figures 4.1, 4.2, A.1).

We then examined geometric changes via intrinsic dimension and dataset entropy (Figures 4.3, A.2). Fine-tuning *compresses* representations (lower ID/entropy) primarily in early-mid layers—again most strongly for multimodal inputs—while deep layers converge to similar, compact regimes across tasks. An analysis of the *within-checkpoint* modality gap (Figures 4.4, 4.5) showed slightly higher text-image alignment for the fine-tuned model in shallow layers, a small reversal in late layers, and task-dependent clustering behavior; taken together, these do not yield a single, unified pattern for the modality gap. Finally, layer patching validated the representational findings: cumulatively replacing early-mid pretrained layers with their fine-tuned counterparts recovers most performance gains on visual QA and image captioning, while patching late layers yields diminishing returns (Figures 4.6, 4.7).

Practically, these results suggest targeting early-mid layers during fine-tuning to capture multimodal integration, while keeping later layers stable to preserve next-token decoding. The patching outcomes also indicate that the multimodal connector learned during feature alignment may suffice for mapping the vision encoder features to the text embedding space of the language model, with most downstream task gains arising from editing the LLM backbone during fine-tuning. Interpretability efforts could also benefit from a focus on the early-mid layers to understand the multimodal integration.

5.2 Limitations and Future Work

This study focused on the impact of visual instruction tuning on the backbone representations of LLaVA, under the following constraints. First, we analyze only the LLaVA-1.5-7B model; the 13B version remains untested, and evaluating it would clarify whether our conclusions generalize. Second, our analysis is restricted to last-token residual-stream embeddings, a broadly used representation in the literature and consistent with our experimental design. Third, the results are limited to COCO-derived data for question answering and image captioning. Extending the analysis to other tasks and datasets would test the robustness of our findings. Finally, all experiments are based on the one-epoch fine-tuned LLaVA model released by the authors. The task-specific outcomes of our modality gap analysis (Figures 4.6 and 4.7) may reflect this constraint, and longer training schedules could yield different results. These limitations define the scope of the present study while pointing toward clear directions for future work on how visual instruction tuning reshapes multimodal LLMs and how to fine-tune them effectively and efficiently.

Appendix A

Appendix

Head-wise Similarity Analysis

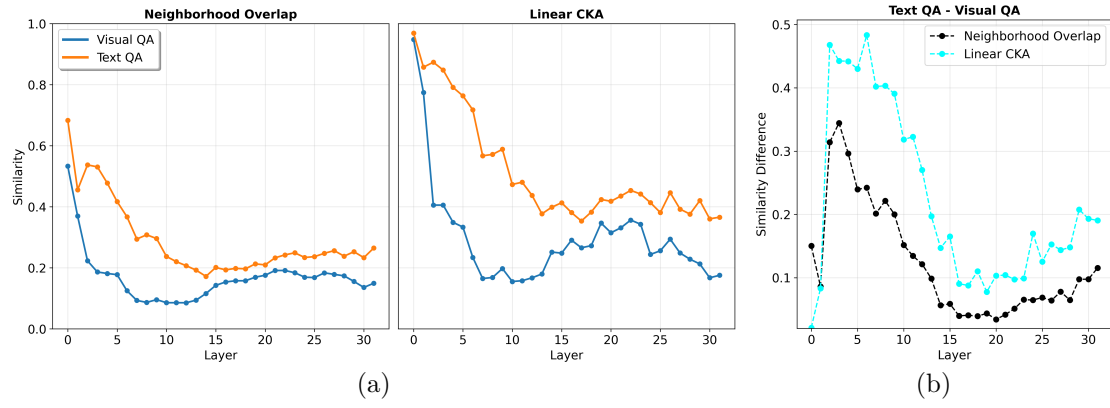


Figure A.1: Similar to Figure 4.1 but the values are the mean of the head-wise neighborhood overlap matrices of the text and multimodal similarities in Figure 4.2.

Dataset Entropy as a Measure of Data Representations Compactness

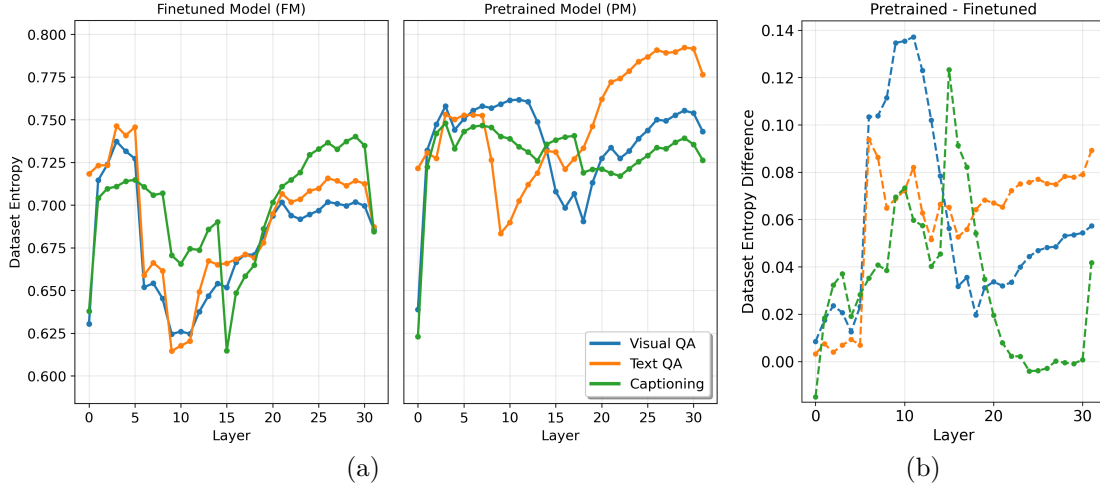


Figure A.2: Dataset entropy. Similar to Figure 4.3. Higher values indicate more diffuse representations, more diverse features and less compact representations.

Question Answering with Sliding-Window Layers Patching

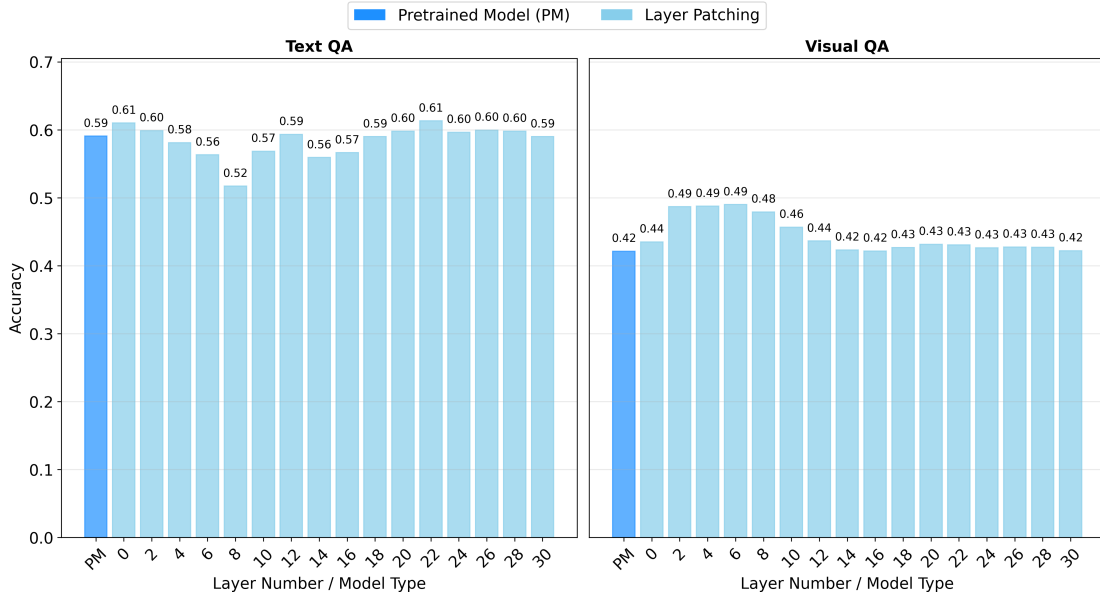


Figure A.3: Sliding-window patching with window size $w = 1$ and step $s = 2$. The x-axis is ℓ (0-indexed; *starting* layer of the patched window). Fine-tuned references: 0.74 (text QA) and 0.66 (visual QA). Similar to Figure 4.6.

Image Captioning with Sliding-Window Layers Patching

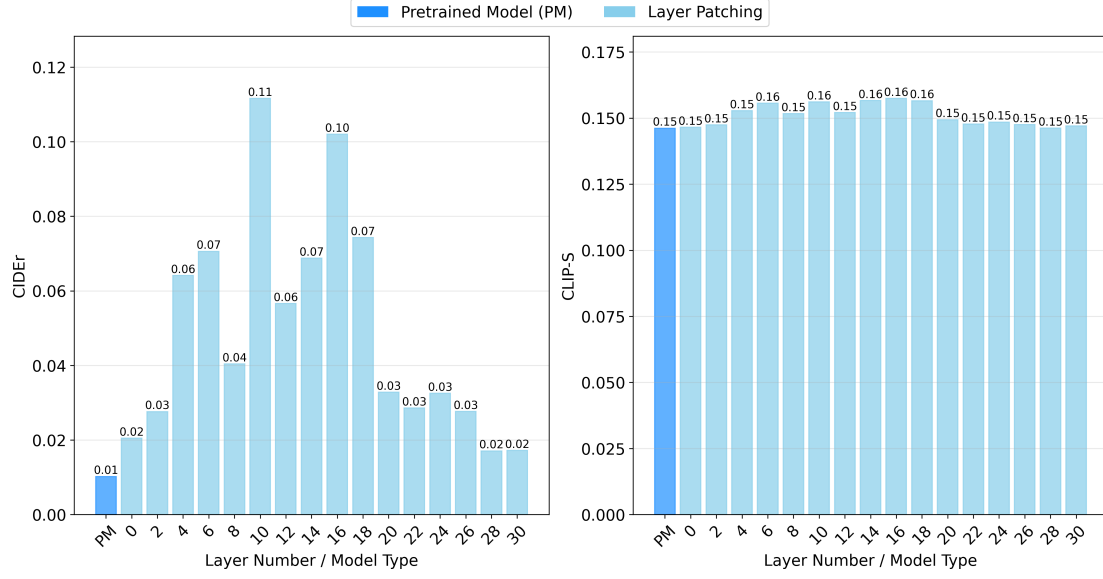


Figure A.4: Sliding-window patching with $w = 1$ (x-axis is the *starting* layer). Fine-tuned references: 0.82 (CIDEr) and 0.22 (CLIP-S). Similar to Figure 4.7.

References

- [1] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao. [Large Language Models: A Survey](#). 2025.
- [2] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen. [A Survey of Large Language Models](#). 2025.
- [3] D. Caffagni, F. Cocchi, L. Barsellotti, N. Moratelli, S. Sarto, L. Baraldi, L. Baraldi, M. Cornia, and R. Cucchiara. “The Revolution of Multimodal Large Language Models: A Survey”. *Findings of the Association for Computational Linguistics: ACL 2024*. Findings 2024. Ed. by L.-W. Ku, A. Martins, and V. Srikumar. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 13590–13618.
- [4] Z. Li, X. Wu, H. Du, F. Liu, H. Nghiem, and G. Shi. [A Survey of State of the Art Large Vision Language Models: Alignment, Benchmark, Evaluations and Challenges](#). 2025.
- [5] H. Liu, C. Li, Q. Wu, and Y. J. Lee. [Visual Instruction Tuning](#). 2023.
- [6] H. Liu, C. Li, Y. Li, and Y. J. Lee. [Improved Baselines with Visual Instruction Tuning](#). 2024.
- [7] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. [Learning Transferable Visual Models From Natural Language Supervision](#). 2021.
- [8] LMSYS Org. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. LMSYS. 2023. URL: <https://lmsys.org/blog/2023-03-30-vicuna/>.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. [Attention Is All You Need](#). 2023.
- [10] R. Bommasani et al. [On the Opportunities and Risks of Foundation Models](#). 2022.
- [11] D. Bahdanau, K. Cho, and Y. Bengio. [Neural Machine Translation by Jointly Learning to Align and Translate](#). 2016.
- [12] A. P. Parikh, O. Täckström, D. Das, and J. Uszkoreit. [A Decomposable Attention Model for Natural Language Inference](#). 2016.

-
- [13] Y. Bengio, A. Courville, and P. Vincent. [Representation Learning: A Review and New Perspectives](#). 2014.
 - [14] R. Sennrich, B. Haddow, and A. Birch. [Neural Machine Translation of Rare Words with Subword Units](#). 2016.
 - [15] K. He, X. Zhang, S. Ren, and J. Sun. [Deep Residual Learning for Image Recognition](#). 2015.
 - [16] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah. A Mathematical Framework for Transformer Circuits. Anthropic. 2021. URL: <https://transformer-circuits.pub/2021/framework/index.html>.
 - [17] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. [LLaMA: Open and Efficient Foundation Language Models](#). 2023.
 - [18] J. L. Ba, J. R. Kiros, and G. E. Hinton. [Layer Normalization](#). 2016.
 - [19] B. Zhang and R. Sennrich. [Root Mean Square Layer Normalization](#). 2019.
 - [20] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. [Finetuned Language Models Are Zero-Shot Learners](#). 2022.
 - [21] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu, and G. Wang. [Instruction Tuning for Large Language Models: A Survey](#). 2024.
 - [22] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. [Training language models to follow instructions with human feedback](#). 2022.
 - [23] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. [Language Models are Few-Shot Learners](#). 2020.
 - [24] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny. [MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models](#). 2023.
 - [25] J. Li, D. Li, S. Savarese, and S. Hoi. [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#). 2023.
 - [26] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. [An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale](#). 2021.

-
- [27] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan. [Flamingo: a Visual Language Model for Few-Shot Learning](#). 2022.
- [28] X. Wang, X. Zhang, Z. Luo, Q. Sun, Y. Cui, J. Wang, F. Zhang, Y. Wang, Z. Li, Q. Yu, Y. Zhao, Y. Ao, X. Min, T. Li, B. Wu, B. Zhao, B. Zhang, L. Wang, G. Liu, Z. He, X. Yang, J. Liu, Y. Lin, T. Huang, and Z. Wang. [Emu3: Next-Token Prediction is All You Need](#). 2024.
- [29] A. Agrawal, J. Lu, S. Antol, M. Mitchell, C. L. Zitnick, D. Batra, and D. Parikh. [VQA: Visual Question Answering](#). 2016.
- [30] J. Lu, J. Yang, D. Batra, and D. Parikh. [Hierarchical Question-Image Co-Attention for Visual Question Answering](#). 2017.
- [31] M. Ren, R. Kiros, and R. Zemel. [Exploring Models and Data for Image Question Answering](#). 2015.
- [32] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, and Y. Bengio. [Show, Attend and Tell: Neural Image Caption Generation with Visual Attention](#). 2016.
- [33] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. [Show and Tell: A Neural Image Caption Generator](#). 2015.
- [34] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár. [Microsoft COCO: Common Objects in Context](#). 2015.
- [35] P. Sharma, N. Ding, S. Goodman, and R. Soricut. “Conceptual Captions: A Cleaned, Hypernymed, Image Alt-text Dataset For Automatic Image Captioning”. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL 2018. Ed. by I. Gurevych and Y. Miyao. Melbourne, Australia: Association for Computational Linguistics, 2018, pp. 2556–2565.
- [36] H. Luo and L. Specia. [From Understanding to Utilization: A Survey on Explainability for Large Language Models](#). version: 2. 2024.
- [37] Z. Lin, S. Basu, M. Beigi, V. Manjunatha, R. A. Rossi, Z. Wang, Y. Zhou, S. Balasubramanian, A. Zarei, K. Rezaei, Y. Shen, B. M. Yao, Z. Xu, Q. Liu, Y. Zhang, Y. Sun, S. Liu, L. Shen, H. Li, S. Feizi, and L. Huang. [A Survey on Mechanistic Interpretability for Multi-Modal Foundation Models](#). 2025.
- [38] Z. Yu and S. Ananiadou. [Understanding Multimodal LLMs: the Mechanistic Interpretability of Llava in Visual Question Answering](#). version: 1. 2024.
- [39] P. Khayatan, M. Shukor, J. Parekh, and M. Cord. [Analyzing Fine-tuning Representation Shift for Multimodal LLMs Steering alignment](#). 2025.

-
- [40] M. Klabunde, T. Schumacher, M. Strohmaier, and F. Lemmerich. “Similarity of Neural Network Models: A Survey of Functional and Representational Measures”. *ACM Computing Surveys* (2025), p. 3728458.
 - [41] D. Doimo, A. Glielmo, A. Ansuini, and A. Laio. [Hierarchical nucleation in deep neural networks](#). 2020.
 - [42] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. [Similarity of Neural Network Representations Revisited](#). 2019.
 - [43] A. Ansuini, A. Laio, J. H. Macke, and D. Zoccolan. [Intrinsic dimension of data representations in deep neural networks](#). 2019.
 - [44] L. Valeriani, D. Doimo, F. Cuturello, A. Laio, A. Ansuini, and A. Cazzaniga. [The geometry of hidden representations of large transformer models](#). 2023.
 - [45] D. Doimo, A. Serra, A. Ansuini, and A. Cazzaniga. [The representation landscape of few-shot learning and fine-tuning in large language models](#). 2024.
 - [46] A. Serra, F. Ortu, E. Panizon, L. Valeriani, L. Basile, A. Ansuini, D. Doimo, and A. Cazzaniga. [The Narrow Gate: Localized Image-Text Communication in Vision-Language Models](#). 2025.
 - [47] W. Liang, Y. Zhang, Y. Kwon, S. Yeung, and J. Zou. [Mind the Gap: Understanding the Modality Gap in Multi-modal Contrastive Representation Learning](#). 2022.
 - [48] F. Role, S. Meyer, and V. Amblard. [Fill the Gap: Quantifying and Reducing the Modality Gap in Image-Text Representation Learning](#). version: 1. 2025.
 - [49] S. Goel, H. Bansal, S. Bhatia, R. A. Rossi, V. Vinay, and A. Grover. [CyCLIP: Cyclic Contrastive Language-Image Pretraining](#). 2022.
 - [50] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. [Measuring Massive Multitask Language Understanding](#). 2021.
 - [51] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. [Llama 2: Open Foundation and Fine-Tuned Chat Models](#). 2023.
 - [52] Y. Wang, H. Ivison, P. Dasigi, J. Hessel, T. Khot, K. R. Chandu, D. Wadden, K. MacMillan, N. A. Smith, I. Beltagy, and H. Hajishirzi. [How Far Can Camels Go? Exploring the State of Instruction Tuning on Open Resources](#). 2023.
 - [53] G. Team et al. [Gemma 3 Technical Report](#). 2025.

-
- [54] MMLU Dataset. Accessed: August 10, 2025. Hugging Face. 2024. URL: <https://huggingface.co/datasets/cais/mmlu>.
 - [55] P. Lu, S. Mishra, T. Xia, L. Qiu, K.-W. Chang, S.-C. Zhu, O. Tafjord, P. Clark, and A. Kalyan. [Learn to Explain: Multimodal Reasoning via Thought Chains for Science Question Answering](#). 2022.
 - [56] F. Li, R. Zhang, H. Zhang, Y. Zhang, B. Li, W. Li, Z. Ma, and C. Li. [LLaVA-NeXT-Interleave: Tackling Multi-image, Video, and 3D in Large Multimodal Models](#). 2024.
 - [57] S. Zhang, Q. Fang, Z. Yang, and Y. Feng. [LLaVA-Mini: Efficient Image and Video Large Multimodal Models with One Vision Token](#). 2025.
 - [58] COCO Karpathy Dataset. Accessed: August 10, 2025. Hugging Face. URL: <https://huggingface.co/datasets/yerevann/coco-karpathy>.
 - [59] A. Karpathy and L. Fei-Fei. [Deep Visual-Semantic Alignments for Generating Image Descriptions](#). 2015.
 - [60] COCO-QA Vietnamese Dataset. Accessed: August 10, 2025. Hugging Face. URL: <https://huggingface.co/datasets/ThucPD/coco-qa-vi>.
 - [61] J. He, M. Rungta, D. Koleczek, A. Sekhon, F. X. Wang, and S. Hasan. [Does Prompt Formatting Have Any Impact on LLM Performance?](#) version: 1. 2024.
 - [62] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#). 2023.
 - [63] R. Vedantam, C. L. Zitnick, and D. Parikh. [CIDEr: Consensus-based Image Description Evaluation](#). 2015.
 - [64] K. Jeong, W. Lee, W. Nam, M. Ma, and P. Kang. [Technical Report of NICE Challenge at CVPR 2024: Caption Re-ranking Evaluation Using Ensembled CLIP and Consensus Scores](#). version: 1. 2024.
 - [65] J. Hessel, A. Holtzman, M. Forbes, R. L. Bras, and Y. Choi. [CLIPScore: A Reference-free Evaluation Metric for Image Captioning](#). 2022.
 - [66] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena” (2023).
 - [67] L. Basile, S. Acevedo, L. Bortolussi, F. Anselmi, and A. Rodriguez. [Intrinsic Dimension Correlation: uncovering nonlinear connections in multimodal representations](#). 2025.
 - [68] M. Davari, S. Horoi, A. Natic, G. Lajoie, G. Wolf, and E. Belilovsky. [Reliability of CKA as a Similarity Measure in Deep Learning](#). 2022.
 - [69] A. Gretton, O. Bousquet, A. Smola, and B. Schölkopf. “Measuring Statistical Dependence with Hilbert-Schmidt Norms”. *Algorithmic Learning Theory*. Ed. by S. Jain, H. U. Simon, and E. Tomita. Berlin, Heidelberg: Springer, 2005, pp. 63–77.

- [70] F. Denti, D. Doimo, A. Laio, and A. Mira. “The generalized ratios intrinsic dimension estimator”. *Scientific Reports* 12.1 (2022). Publisher: Nature Publishing Group, p. 20005.
- [71] E. Facco, M. d’Errico, A. Rodriguez, and A. Laio. “Estimating the intrinsic dimension of datasets by a minimal neighborhood information”. *Scientific Reports* 7.1 (2017). Publisher: Nature Publishing Group, p. 12140.
- [72] A. Glielmo, I. Macocco, D. Doimo, M. Carli, C. Zeni, R. Wild, M. d’Errico, A. Rodriguez, and A. Laio. “DADApY: Distance-based Analysis of DATA-manifolds in Python”. *Patterns* 3.10 (2022), p. 100589.
- [73] M. d’Errico, E. Facco, A. Laio, and A. Rodriguez. [Automatic topography of high-dimensional data sets by non-parametric Density Peak clustering](#). 2021.
- [74] A. Rosenberg and J. Hirschberg. “V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure”. *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*. Ed. by J. Eisner. Prague, Czech Republic: Association for Computational Linguistics, 2007, pp. 410–420.
- [75] B. Chen, Z. Zhang, N. Langrené, and S. Zhu. [Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review](#). version: 5. 2024.
- [76] H. Liu. LLaVA Model Zoo. Accessed: August 25, 2025. GitHub. URL: https://github.com/haotian-liu/LLaVA/blob/main/docs/MODEL_ZOO.md.
- [77] O. Skean, M. R. Arefin, D. Zhao, N. Patel, J. Naghiyev, Y. LeCun, and R. Shwartz-Ziv. [Layer by Layer: Uncovering Hidden Representations in Language Models](#). 2025.